Computing skeletal decomposition in time linear in the skeleton

Beniamino Accattoli, Francesco Magliocca, Loïc Peyrot, Claudio Sacerdoti Coen

arxiv:2505.09242v3; CC BY4.0

Editorial note. Fixed arXivv3 independently carries CC BY4.0. Author article source and appendices preserved; the repository software is neither downloaded nor included. Native source archive includes the author-bundled LPPL utility and license; license retained separately.

Editorial note. All native mathematical input files and TikZ diagrams are mechanically flattened with comments removed. Author source archive is unchanged. Appendix implementation examples and example machine run are not needed for these counted proofs; definitions, main sections and every mathematical proof appendix are retained.

Editorial note. No correctness certification or formal execution performed. Only stated counted theorem is admitted; support lemmas and direct corollaries remain uncounted.

Editorial scope. Complete author material collection for one structural theorem; author retains responsibility for the mathematics.

Allowed standard prerequisites (editorial).

- Lambda calculus, explicit substitutions and random-access machine model. Complete source sections Background, Skeletal CbNeed and abstract-machine conventions retained.

1 Introduction

Call-by-need evaluation of the λ-calculus was introduced by Wadsworth in 1971 as an optimization of the untyped λ-calculus [41]. It combines the advantages of both call-by-name and call-by-value: it avoids diverging on unused arguments, as in call-by-name, and when arguments are evaluated they do so only once, as in call-by-value. Such a combination is trickier to specify than call-by-name or call-by-value. In particular, Wadsworth's original presentation via graph reduction was not easy to manage.

In the 1990s, call-by-need was adopted by the then new Haskell language, and more manageable term-based calculi were developed by Launchbury [35] and Ariola et al. [21], and extended with non-determinism by Kutzner and Schmidt-Schauß [34]. Additionally, Sestoft studied abstract machines for call-by-need [38]. In 2014, Accattoli et al. introduced the *CbNeed linear substitution*

calculus (CbNeed LSC) [4], a simplification of Ariola et al.'s calculus resting on the *at a distance* approach to explicit substitutions by Accattoli and Kesner [14]. It became a reference presentation, used in many recent studies [31, 24, 5, 33, 26, 13, 32, 25, 17, 16]. As far as this paper is concerned, the study in [4] is particularly relevant: beyond introducing the CbNeed LSC that we shall use, it also relates it with abstract machines for CbNeed, building a neat bisimulation between the two, deemed *distillation*.

Skeletal Call-by-Need. CbNeed was introduced together with a further optimization usually called *fully lazy sharing* and here rather referred to as *Skeletal CbNeed* (because *fully lazy sharing* is not really descriptive). The idea is to refine the duplication of a value v so as to duplicate only the *skeleton* of v while keeping its *flesh* shared, thus ending up sharing more than in ordinary call-by-need. As an example, if $v = \lambda x.\lambda y.zzx(yz)$ then the skeleton of v is $\mathtt{skel}(v) := \lambda x.\lambda y.wx(yz)$ and its flesh is the context $\mathtt{flesh}(v) := \mathsf{let}\ w = zz\ \mathsf{in}\ \langle\cdot\rangle$. That is, the flesh collects and removes the maximal (non-variable) sub-terms of v that are *free*, *i.e.* such that none of their free variables is captured in v.

Skeletal CbNeed is less studied than CbNeed because it is even trickier to define, as it requires to compute the skeletal decomposition of a value (that is, the split into skeleton and flesh). Early works by Turner [40], Hughes [30], and Peyton Jones [36] focus on implementative aspects. In the last ten years or so, various works provided operational insights. Balabonski developed an impressive theoretical analysis of Skeletal CbNeed, showing the equivalence of various presentations [22] and studying the relationship with Lévy's optimality [23]. Soon after that, Gundersen et al. showed that the *atomic λ-calculus*, a λ-calculus with sharing issued from deep inference technology, naturally specifies the duplication of the skeleton [29]. Next, Kesner et al. [32] merged the insights of the CbNeed LSC and the atomic λ-calculus obtaining a Skeletal CbNeed LSC, that is, a presentation at a distance of Skeletal CbNeed.

This Paper. The present work extends the operational research line of Balabonski, Gundersen et al., and Kesner et al. with cost analyses of Kesner et al.'s Skeletal CbNeed LSC, aiming at closing the gap with some of the earlier implementative studies at the same time. Generally speaking, we aim at adding Skeletal CbNeed to the list of concepts covered by the theory of cost-based analyses of abstract machines and sharing mechanisms by Accattoli and co-authors [4, 10, 19, 5, 7, 27, 12, 11, 8, 15]. We provide two analyses.

First Analysis: Exponential Time and Space Speed-Ups. The first analysis shows that in some cases Skeletal CbNeed is considerably more efficient than CbNeed. For that, we exhibit a family of λ-terms $\{t_n\}_{n\in\mathbb{N}}$ such that t_n evaluates in $\Omega(2^n)$ time and space in CbNeed, while requiring only $\mathcal{O}(n)$ space and time in Skeletal CbNeed. The literature only shows examples of single λ-terms where Skeletal CbNeed takes a few steps less than CbNeed, but never proves any asymptotic speed-up nor deals with space. Lastly, proving that our family evaluates within the given bounds requires a fine, non-trivial analysis of evaluation sequences.

Second Analysis: Bi-Linear Overhead. The second analysis shows that Skeletal CbNeed can be implemented efficiently, with an overhead that is *bi-linear*, *i.e.* linear in both the number of β-steps and the size of the initial term. This time, the same is true for CbNeed. The insight is that computing skeletons does require additional implementative efforts, but it comes at no asymptotic price. This work was indeed triggered by showing that the operational reconstruction of the skeleton at work in Gundersen et al. [29] and Kesner et al. [32] can be implemented in linear time. Their specifications rest on side conditions about variables in sub-terms that lead to non-linear overhead, if implemented literally. In fact, the linear time reconstruction of skeletons by Shivers and Wand [39], from 2010, avoids the side conditions and pre-dates the recent works. The insightful study in [39], however, is technical and not well-known. Here we simplify it, recasting it in the context of abstract machines.

Efficient Skeletal Decompositions, or Shivers and Wand Reloaded. Shivers and Wand present their ideas using graph-reduction for λ-terms. They reconstruct the skeletal decomposition via a visit of the value to skeletonize seen as a graph, based on two key points:

1. *Bi-directional edges*: all the edges of their graphs can be traversed in *both* directions. When term graphs are seen as mathematical objects, edges are often directed but the theoretical study can also look at edges in reverse. At the implementative level, however, things are different. For a parsimonious use of space, most graph-based implementations (*e.g.* the proof nets inspired ones in [5, 8, 20]) indeed restrict some edges to be traversed in one direction only (namely, parent to children), thus sparing some pointers. Therefore, bi-directional edges are an unusual approach, usually considered redundant.

2. *Abstractions, occurrences, and upward reconstruction*: in most graphical representations, there are edges between abstractions and the occurrences of their bound variables, usually directed only from the occurrences to the abstraction. Shivers and Wand's insight is that, with bi-directional edges, one can move from the abstraction back to the occurrences of the variable and then reconstruct the skeleton by visiting upward from the occurrences.

Shivers and Wand's study is graph-based and low-level, interleaved with code snippets. Here, we provide a new neat presentation of their algorithm as a simple high-level rewriting system over terms, circumventing graphs and low-level details. The reformulation of their study is the main technical innovation of this paper. While the key concepts are due to Shivers and Wand, we believe that our new presentation is a notable contribution in that it allows their concepts to blossom, simplifying their study and making them more widely accessible.

Distillation. We then smoothly lift the distillation-based study of CbNeed by Accattoli et al. [4] to Skeletal CbNeed, providing an abstract machines using the reloaded algorithm for skeletal decompositions as a black-box. The new abstract machine is then shown to be implementable within a bi-linear overhead, completing the second analysis.

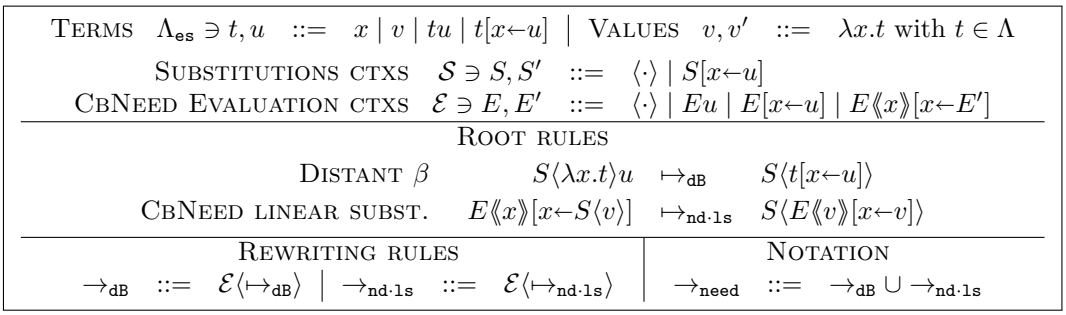


Figure 1: The Call-by-Need Linear Substitution Calculus (CbNeed LSC).

The overall insight is that the apparently linear space inefficiency of adding pointers for bi-directional traversals of terms/graphs enables optimizations—namely, skeletal duplications—that can bring, in some cases, exponential speed-ups for both time and space.

Implementation. We provide an OCaml implementation of the abstract machine, and in particular of the reloaded skeleton reconstruction algorithm it rests upon. The implementation is there to validate the complexity analyses, as well as to integrate the low-level aspect of Shivers and Wand's study. The implementation is discussed in Appendix G.

2 Background: Call-by-Need

In this section, we recall CbNeed, presented as a strategy of the CbNeed linear substitution calculus (shortened to LSC). For the sake of simplicity, we do not distinguish here between the strategy and the calculus, and use *CbNeed LSC* to refer to both.

Accattoli and Kesner's LSC [1, 6] is a micro-step λ-calculus with explicit substitutions. *Micro-step* means that substitutions act on one variable occurrence at a time, rather than *small-step*, that is, on all occurrences at the same time. The CbNeed LSC was introduced by Accattoli et al. [4] as a simplified variant of Ariola et al.'s presentation of CbNeed [21]. The simplification is the use of rewriting rules *at a distance*—that is, adopting contexts in the root rules—that allow one to get rid of the commuting rewriting rules of Ariola et al.

Terms and Contexts. The set of terms of the λ-calculus is noted Λ. The CbNeed LSC is defined in Fig. 1. The set of terms of the CbNeed LSC is noted $\Lambda_{\mathtt{es}}$. They add *explicit substitutions* $t[x\leftarrow u]$ (shortened to ESs) to λ-terms, that is a more compact notation for $\mathsf{let}\ x = u\ \mathsf{in}\ t$, but where the order of evaluation between t and u is a priori not fixed; evaluation contexts shall fix it. The set $\mathtt{fv}(t)$ of *free* variables is defined as expected, in particular, $\mathtt{fv}(t[x\leftarrow u]) := (\mathtt{fv}(t)\setminus\{x\})\cup\mathtt{fv}(u)$. Both $\lambda x.t$ and $t[x\leftarrow u]$ bind x in t, and terms are identified up to α-renaming. A term t is *closed* if $\mathtt{fv}(t)=\emptyset$, *open* otherwise. Meta-level capture-avoiding substitution is noted $t\{x\leftarrow u\}$. The size $|t|$ of t is the number of its construct.

Note that values are only abstraction; this choice is standard in the literature on CbNeed. Note that, moreover, their bodies are λ-terms *without* ESs. This is in order to avoid dealing with calculating the skeleton of ESs in the next section. It is a harmless choice because evaluation (that creates ESs) never enters inside abstraction.

Contexts are terms with exactly one occurrence of the *hole* $\langle\cdot\rangle$, an additional constant, standing for a removed sub-term. We shall heavily use two notions of contexts: *substitution contexts* S, that are lists of ESs, and evaluation contexts E.

The main operation about contexts is *plugging* $E\langle t\rangle$ where the hole $\langle\cdot\rangle$ in context E is replaced by t. Plugging, as usual with contexts, can capture variables; for instance $((\langle\cdot\rangle t)[x{\leftarrow}u])\langle x\rangle = (xt)[x{\leftarrow}u]$. We write $E\langle\!\langle t\rangle\!\rangle$ when we want to stress that the context E does not capture the free variables of t.

Rewriting Rules. The reduction rules of the CbNeed LSC are slightly unusual as they use *contexts* both to allow one to reduce redexes located in sub-terms (via evaluation contexts E), which is standard, *and* to define the redexes themselves (via both substitution S and evaluation contexts E), which is less standard. This approach is called *at a distance* and related to cut-elimination on proof nets; see Accattoli [2, 3] for the link with proof nets. The notion of evaluation context E used for CbNeed is exactly the one by Ariola et al. [21], that extend call-by-name evaluation contexts with the production $E\langle\!\langle x\rangle\!\rangle[x{\leftarrow}E']$, which—by enterings ESs—is what enables the memoization aspect of CbNeed.

The *distant beta rule* $\mapsto_{\mathtt{dB}}$ is essentially the β-rule, except that the argument goes into a new ES, rather then being immediately substituted, and that there can be a substitution context S in between the abstraction and the argument. Example: $(\lambda x.y)[y{\leftarrow}t]u \mapsto_{\mathtt{m}} y[x{\leftarrow}u][y{\leftarrow}t]$. One with on-the-fly α-renaming is $(\lambda x.y)[y{\leftarrow}t]y \mapsto_{\mathtt{m}} z[x{\leftarrow}y][z{\leftarrow}t]$.

The *linear substitution rule by need* $\mapsto_{\mathtt{nd{\cdot}ls}}$ replaces a single variable occurrence by a copy of the value in the ES, commuting the substitution context around the value (if any) out of the ES. Example: $(xx)[x{\leftarrow}\mathtt{I}[y{\leftarrow}t]] \mapsto_{\mathtt{nd{\cdot}ls}} (\mathtt{I}x)[x{\leftarrow}\mathtt{I}][y{\leftarrow}t]$.

The use of substitution contexts S in the root rules is what allows one to avoid the two commuting rules of Ariola et al. [21], namely (rephrasing their let-expressions as ESs) $t[x{\leftarrow}s]u \mapsto_{let\text{-}C} (tu)[x{\leftarrow}s]$ and $t[y{\leftarrow}u[x{\leftarrow}s]] \mapsto_{let\text{-}A} t[y{\leftarrow}u][x{\leftarrow}s]$.

The two root rules $\mapsto_{\mathtt{dB}}$ and $\mapsto_{\mathtt{nd{\cdot}ls}}$ are then closed by evaluation contexts E. In Fig. 1, we use the compact notation $\to_{\mathtt{dB}} := \mathcal{E}\langle\mapsto_{\mathtt{dB}}\rangle$ to denote that $\to_{\mathtt{dB}}$ is defined as $E\langle t\rangle \to_{\mathtt{dB}} E\langle u\rangle$ if $t \mapsto_{\mathtt{dB}} u$, and similarly for the other rule.

3 Skeletal Call-by-Need

In this section, we present our version of Skeletal CbNeed, which is a very minor variant of the one by Kesner et al. [32] (the difference is discussed at the end of the section), itself defined by tweaking the CbNeed LSC via the notion of skeleton.

The basic idea of skeletal CbNeed is that every value $v := \lambda x.t$ can be split in two: the skeleton $\lambda x.u$, which is a sort of sub-term of t, and the flesh S, which is a substitution context collecting the maximal sub-terms of v that do

TERMS	$\Lambda_{\mathtt{sk}} \ni t, u$	$::=$	$x \mid v \mid tu \mid t[x{\leftarrow}u] \mid t\langle x{\leftarrow}v\rangle$ with $v = \mathtt{skel}(v)$
VALUES	v, v'	$::=$	$\lambda x.t$ with $t \in \Lambda$
SUBSTITUTIONS CTXS	$\mathcal{S}_{\mathtt{sk}} \ni S, S'$	$::=$	$\langle\cdot\rangle \mid S[x{\leftarrow}u] \mid S\langle x{\leftarrow}v\rangle$
SKELETAL EVAL. CTXS	$\mathcal{E}_{\mathtt{sk}} \ni E, E'$	$::=$	$\langle\cdot\rangle \mid Eu \mid E[x{\leftarrow}u] \mid E\langle x{\leftarrow}v\rangle \mid E\langle\!\langle x\rangle\!\rangle[x{\leftarrow}E']$

ROOT RULES

DISTANT β	$S\langle\lambda x.t\rangle u$	$\mapsto_{\mathtt{dB}}$	$S\langle t[x{\leftarrow}u]\rangle$
SKELETONIZATION	$E\langle\!\langle x\rangle\!\rangle[x{\leftarrow}S\langle v\rangle]$	$\mapsto_{\mathtt{sk}}$	$S\langle S'\langle E\langle\!\langle x\rangle\!\rangle\langle x{\leftarrow}v'\rangle\rangle\rangle$ with $\mathtt{skdec}(v) = (v', S')$
SKELETAL SUBST.	$E\langle\!\langle x\rangle\!\rangle\langle x{\leftarrow}v\rangle$	$\mapsto_{\mathtt{ss}}$	$E\langle\!\langle v\rangle\!\rangle\langle x{\leftarrow}v\rangle$

REWRITING RULES

$$\rightarrow_{\mathtt{sk{\cdot}dB}} ::= \mathcal{E}_{\mathtt{sk}}\langle\mapsto_{\mathtt{dB}}\rangle \qquad \rightarrow_{\mathtt{sk}} ::= \mathcal{E}_{\mathtt{sk}}\langle\mapsto_{\mathtt{sk}}\rangle$$
$$\rightarrow_{\mathtt{ss}} ::= \mathcal{E}_{\mathtt{sk}}\langle\mapsto_{\mathtt{ss}}\rangle$$

NOTATIONS

$\rightarrow_{\mathtt{sk{\cdot}need}} ::= \rightarrow_{\mathtt{sk{\cdot}dB}} \cup \rightarrow_{\mathtt{sk}} \cup \rightarrow_{\mathtt{ss}}$

$[\![x{\leftarrow}t]\!]$ stands for either $[x{\leftarrow}t]$ or $\langle x{\leftarrow}v\rangle$

Figure 2: The skeletal CbNeed strategy $\rightarrow_{\mathtt{sk{\cdot}need}}$.

not depend on x. The flesh is extracted from v before duplicating it, as to avoid duplicating the code (and possibly redexes) of the flesh, thus increasing sharing. Some definitions are in order.

Free Sub-Terms. Extractable sub-terms are called *free sub-terms*, defined next. We actually need a parametrized notion of free sub-term, which is relative to a set of variables $\mathtt{V}$ that are supposed not to occur in the sub-term.

Definition 3.1 (Maximally free sub-terms)**.** *Let $t \in \Lambda$ and $\mathtt{V}$ be a set of variables. A sub-term u of t is (the term part of) a decomposition $t = C\langle u\rangle$ for some context C; additionally, u is* $\mathtt{V}$-free in t *if no variable in $\mathtt{fv}(u)$ is captured by C and if $\mathtt{fv}(u) \cap \mathtt{V} = \emptyset$, and* maximally $\mathtt{V}$-free *if it is not a strict sub-term of any other $\mathtt{V}$-free sub-term in t.*

Skeleton. The skeletal decomposition of a term t is the splitting of t into its maximal free sub-terms on one side, collected as a substitution context S called *the flesh*, and what is left of t after the removal, that is, its *skeleton*. As for free sub-terms, we rather define parametrized notions of skeletal decomposition and skeleton. Then, we specialize the definitions for values.

Definition 3.2 ((Relative) Skeleton)**.** *Let $\mathtt{V}$ be a set of variables. The skeletal decomposition of t relative to a set of variables $\mathtt{V}$ is the pair (u, S) of an ordinary λ-term u and a substitution context S defined as $\mathtt{skdec}^{\mathtt{V}}(t) := (x, \langle\cdot\rangle[x{\leftarrow}t])$ with x fresh, if $\mathtt{fv}(t) \cap \mathtt{V} = \emptyset$ and t is not a variable, otherwise:*

$$\begin{array}{rcll} \mathtt{skdec}^{\mathtt{V}}(x) & := & (x, \langle\cdot\rangle) & \\ \mathtt{skdec}^{\mathtt{V}}(\lambda x.u) & := & (\lambda x.s, S) & \text{where } \mathtt{skdec}^{\mathtt{V}\cup\{x\}}(u) = (s, S) \\ \mathtt{skdec}^{\mathtt{V}}(us) & := & (rp, S'\langle S\rangle) & \text{where } \mathtt{skdec}^{\mathtt{V}\cup\{x\}}(u) = (r, S) \text{ and } \mathtt{skdec}^{\mathtt{V}\cup\{x\}}(s) = (p, S') \end{array}$$

The skeleton, the flesh, and the skeletal decomposition of $\lambda x.t$ are defined respectively as $\mathtt{skel}(\lambda x.t) := \lambda x.u$, $\mathtt{flesh}(\lambda x.t) := S$, and $\mathtt{skdec}(\lambda x.t) := (\mathtt{skel}(\lambda x.t), \mathtt{flesh}(\lambda x.t))$, where $\mathtt{skdec}^{\{x\}}(t) = (u, S)$.

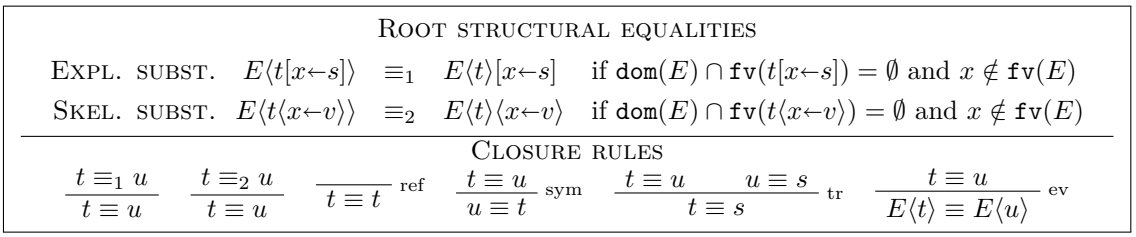


Figure 3: Structural equivalence $\equiv$.

Skeletal CbNeed. We adopt the presentation of skeletal CbNeed by Kesner et al. [32] (refered to as *fully lazy CbNeed* by them), that adds a new *skeletal (explicit) substitution* $t\langle x\leftarrow v\rangle$ containing *skeletal values*, that is, values that are equal to their skeleton. The language of terms and the strategy is defined in Fig. 2. The new set of terms is noted $\Lambda_{\mathtt{sk}}$.

Notation: when we need to treat explicit and skeletal substitution together, we use the notation $[\![x\leftarrow t]\!]$, which stands for either $[x\leftarrow t]$ or (with a slight abuse) $\langle x\leftarrow v\rangle$.

The CbNeed substitution rule $\rightarrow_{\mathtt{nd\cdot ls}}$ for a redex $E\langle\!\langle x\rangle\!\rangle[x\leftarrow S\langle v\rangle]$ is refined via two rules. The first *skeletonization* rule $\rightarrow_{\mathtt{sk}}$ does three further mini tasks at once: firstly, it decomposes v in its skeleton v' and its flesh S' (thus obtaining $E\langle\!\langle x\rangle\!\rangle[x\leftarrow S\langle S'\langle v\rangle\rangle]$); secondly, it re-organizes the term commuting S and S' out of the ES (obtaining $S\langle S'\langle E\langle\!\langle x\rangle\!\rangle[x\leftarrow v']\rangle\rangle$); thirdly, it turns the ES into a skeletal substitutions, finally producing $S\langle S'\langle E\langle\!\langle x\rangle\!\rangle\langle x\leftarrow v'\rangle\rangle\rangle$.

The second *skeletal substitution* rule $\rightarrow_{\mathtt{ss}}$ simply replaces x with v'.

Note that evaluation contexts E are adapted to include skeletal substitutions, but that they do not enter inside them, since their content is always a value. The distant β rule for Skeletal CbNeed is noted $\rightarrow_{\mathtt{sk\cdot dB}}$ to distinguish it from the one for CbNeed. The Skeletal CbNeed reduction $\rightarrow_{\mathtt{sk\cdot need}}$ is the union of $\rightarrow_{\mathtt{sk\cdot dB}}$, $\rightarrow_{\mathtt{sk}}$, and $\rightarrow_{\mathtt{ss}}$.

The next proposition is an easy adaptation of the one for CbNeed in Accattoli et al. [4].

Proposition 3.3. *The reduction $\rightarrow_{\mathtt{sk\cdot need}}$ is deterministic.* Proof at p. 21

Structural Equivalence. To study the relationship with abstract machines, we shall need the concept of structural equivalence, defined in Fig. 3. The concept is standard for λ-calculi with ESs at a distance, and it also standard to use it in relationship with abstract machines, as systematically done by Accattoli et al. [4]. Intuitively, structural equivalence allows one to move explicit/skeletal substitutions around because the move does not change the behaviour of the term. This fact is captured by the following strong bisimulation property, proved by an immediate adaptation of the similar property in [4].

Proposition 3.4 ($\equiv$ is a strong bisimulation)**.** *Let $a\in\{\mathtt{dB},\mathtt{sk},\mathtt{ss}\}$. If $t\equiv u$ and $t\rightarrow_a t'$ then there exists u' such that $u\rightarrow_a u'$ and $t'\equiv u'$.*

Difference with Kesner et al's Skeletal CbNeed [32]. There is a small difference between our definition of Skeletal CbNeed and Kesner et al.'s, namely in the definition of skeletons. For us, the skeletal decomposition of, say,

$v := \lambda x.yx(zz)$ is $(\lambda x.yxw, \langle\cdot\rangle[w{\leftarrow}zz])$, while for them it is $(\lambda x.y'xw, \langle\cdot\rangle[y'{\leftarrow}y][w{\leftarrow}zz])$, that is, they *flesh-out* also single variable occurrences while we do *not*, obtaining a slightly more parsimonious decomposition.

This difference, however, has essentially no impact on the theory, nor on the asymptotic costs. In particular, the operational equivalence of Skeletal CbNeed with respect to CbNeed and CbN is preserved. The proof of Kesner et al., indeed, works for all decompositions such that substituting the flesh into the skeleton recovers the original value; our decomposition has this property. Therefore, we can import the following result.

Theorem 3.5 (Operational equivalence [32])**.** *Two λ-terms t and u are CbN observational equivalent if and only if they are Skeletal CbNeed observational equivalent.*

4 Skeletal CbNeed Can Bring an Exponential Asymptotic Speed-Up

In this section, we show that Skeletal CbNeed can be both exponentially faster and use exponentially less space than CbNeed, taking as reference cost models (following the terminology of Accattoli et al. [15]):

- *Abstract time*, that is the number of (distant) β-steps;
- *Ink space*, that is, the maximum size of the involved terms.

By smoothly adapting results in the literature [37, 9, 28], these can be proved to be reasonable cost models for time for CbNeed and for (super-)linear space for both CbNeed and Skeletal CbNeed. In fact, the relevant part of proving that abstract time is reasonable for Skeletal CbNeed shall be our own Theorem 8.5 at the end of the paper.

We prove the result by defining a family of terms $\{t_n\}_{n\in\mathbb{N}}$ and showing that t_n evaluates in an exponential number of $\mathtt{dB}$ steps and producing terms of exponential size in CbNeed, while it uses only a linear number of $\mathtt{sk}{\cdot}\mathtt{dB}$ steps and terms of linear size in Skeletal CbNeed.

The Family. We shall use the abbreviations $\mathtt{I} := \lambda x.x$ and $\gamma := \lambda y.\lambda z.y\mathtt{I}(y\mathtt{I})z$. Next, we define an auxiliary family $\{u_n\}_{n\in\mathbb{N}}$ as follows: $u_0 := \mathtt{I}$ and $u_{n+1} := \gamma u_n$. Lastly, the actual family is defined as $t_n := (\lambda x.x\mathtt{I}(x\mathtt{I}))u_n$.

Our result is that the term t_n evaluates in $\mathcal{O}(n)$ $\to_{\mathtt{sk}\cdot\mathtt{dB}}$-steps (namely $6n+4$ steps) with skeletal CbNeed and $\Omega(2^n)$ $\to_{\mathtt{dB}}$-steps (namely $8\times 2^n + n - 4$ steps) with CbNeed. This is shown via quite technical statements. Firstly, note that $t_n \to_{\mathtt{dB}} (x\mathtt{I}(x\mathtt{I}))[x{\leftarrow}u_n]$; it is actually for $(x\mathtt{I}(x\mathtt{I}))[x{\leftarrow}u_n]$ that we shall prove bounds. These bounds are expressed stating that there are families of evaluation contexts $S^{\mathtt{nd}}_n$ and $S^{\mathtt{sk}}_n$ (one for CbNeed and one for Skeletal CbNeed) such that $(x\mathtt{I}(x\mathtt{I}))[x{\leftarrow}u_n]$ reduces to $S^{\mathtt{nd}}_n\langle\mathtt{I}\rangle$ and $S^{\mathtt{sk}}_n\langle\mathtt{I}\rangle$ (which is a normal form in both cases) in $6n+3$ and $8\times 2^n + n - 3$ steps respectively. The results about space are obtained by showing that $S^{\mathtt{nd}}_n$ has size exponential in n, while $S^{\mathtt{sk}}_n$ is linear in n.

Evaluation in Skeletal CbNeed. The skeletal case is, perhaps surprisingly, the easiest case for which one can find an inductive invariant leading to the bound.

Lemma 4.1. *Let $n \in \mathbb{N}$ and $s_n := (x\mathtt{I}(x\mathtt{I}))[x \leftarrow u_n]$. Then there is a substitution context $S_n^{\mathtt{sk}}$ and an evaluation $\mathtt{e}_n : s_n \to^*_{\mathtt{sk\cdot need}} S_n^{\mathtt{sk}}\langle \mathtt{I} \rangle$ such that $|\mathtt{e}_n|_{\mathtt{sk\cdot dB}} = 6n+3$ and $|S_n^{\mathtt{sk}}| = \mathcal{O}(n)$.* Proof at p. 22

Evaluation in CbNeed. For the non-skeletal case, the bound is formulated via two families of substitution contexts. For $n \in \mathbb{N}$, the first family is given by $S_0 := \langle\cdot\rangle[x_0 \leftarrow \mathtt{I}]$ and $S_{n+1} := S_n\langle\langle\cdot\rangle[x_{n+1} \leftarrow s_{n+1}]\rangle$ with $s_{n+1} := \lambda y_n.x_n\mathtt{I}(x_n\mathtt{I})y_n$. The second family $S_n^{\mathtt{nd}}$ is defined in the proof of the lemma, itself relying on an auxiliary lemma in Appendix B. The statement is parametrized by an evaluation context E for the induction to go through, but the relevant case is the one with E empty.

Lemma 4.2. *Let $n \in \mathbb{N}$ and E be an evaluation context. Then there exist a substitution context $S_n^{\mathtt{nd}}$ and an evaluation $\mathtt{e}_n : E\langle\!\langle x_n\mathtt{I}(x_n\mathtt{I})\rangle\!\rangle[x_n \leftarrow u_n] \to^*_{\mathtt{need}} S_n\langle E\langle\!\langle S_n^{\mathtt{nd}}\langle \mathtt{I}\rangle\rangle\!\rangle\rangle$ such that $|\mathtt{e}_n|_{\mathtt{dB}} = 8 \times 2^n + n - 5$ and $|S_n^{\mathtt{nd}}| = \Omega(2^n)$.* Proof at p. 25

The next statement sums up the results, and is based on the result at the end of the paper stating that Skeletal CbNeed can be implemented in bilinear time, that is, linear in the size $|t_n|$ of the initial term and in the number of $\mathtt{sk\cdot dB}$ steps (Theorem 8.5).

Theorem 4.3 (Instance of exponential skeletal speed-up for both time and space)**.** *There is a family $\{t_n\}_{n\in\mathbb{N}}$ of λ-terms such that t_n normalizes in $\mathcal{O}(n)$ abstract time and $\mathcal{O}(n)$ ink space with Skeletal CbNeed and $\Omega(2^n)$ abstract time and $\Omega(2^n)$ ink space with CbNeed.*

Proof.

- For CbNeed, by Lemma 4.2 there is an evaluation sequence:

  $$t_n = (\lambda x.x\mathtt{I}(x\mathtt{I}))u_n \to_{\mathtt{dB}} (x\mathtt{I}(x\mathtt{I}))[x \leftarrow u_n] \to^*_{\mathtt{need}} S_n\langle S_n^{\mathtt{nd}}\langle \mathtt{I}\rangle\rangle$$

  taking $k_n := 8 \times 2^n + n - 4 = \Omega(2^n)$ $\mathtt{dB}$-steps and such that $|S_n^{\mathtt{nd}}| = \Omega(2^n)$.

- For Skeletal CbNeed, by Lemma 4.1 there is an evaluation sequence:

  $$t_n = (\lambda x.x\mathtt{I}(x\mathtt{I}))u_n \to_{\mathtt{sk\cdot dB}} (x\mathtt{I}(x\mathtt{I}))[x \leftarrow u_n] \to^*_{\mathtt{sk\cdot need}} S_n^{\mathtt{sk}}\langle \mathtt{I}\rangle$$

  taking $k_n := 6n + 4 = \mathcal{O}(n)$ $\mathtt{sk\cdot dB}$-steps and such that $|S_n^{\mathtt{sk}}| = \mathcal{O}(n)$. Note that the calculus has no garbage collection rule, so the size of terms diminishes only by $\mathtt{sk\cdot dB}$ steps, and of a constant amount (a $\mathtt{sk\cdot dB}$ step removes two constructs and adds one). Therefore, the maximum size of terms is bounded by the size of the final term plus the number of steps. That is, the space cost is $\mathcal{O}(n)$. □

MARKED TERMS	$\Lambda_\circ \ni m, n$	$::=$	$x \mid x^\circ \mid \lambda x.m \mid \lambda^\circ x.m \mid mn \mid m \circ n \mid \overline{m}^{\Uparrow}$
MARKED CONTEXTS	$\mathcal{M} \ni M$	$::=$	$\langle\cdot\rangle \mid \lambda x.M \mid \lambda^\circ x.M \mid Mn \mid mM$
			$\mid M \circ n \mid m \circ M \mid \overline{M}^{\Uparrow}$

Figure 4: Marked terms and contexts.

5 A Rewriting-Based Algorithm for Computing the Skeleton

In this section, we first define *marked* terms and a marked reformulation of skeletal decompositions. Then, we use the marked setting to give a graphically-inspired rewriting system computing the skeletal decomposition of a value, giving a neat new presentation of ideas first developed by Shivers and Wand [39]. The rewriting system formalizes an algorithm. It is presented as a rewriting system as to smoothly manage it as an operational semantics notion.

Marked Skeleton. Marked terms and contexts are defined in Fig. 4. The idea is to incrementally mark with a tag $\circ$ the constructs belonging to the skeleton during a visit of the term which is allowed to jump from an abstraction to the occurrences of the variables, since these are usually connected in a graphical representation.

Constructs of the λ calculus are then either the usual ones (referred to as *unmarked*) or marked with $\circ$, which means that the constructor is part of the skeleton. Additionally, there is an extra *up* construct $\overline{t}^{\Uparrow}$ that shall be used to indicate the frontier of the visit of the term.

In the marked setting, we can recast skeletal decompositions by marking as white the constructs belonging to the skeleton and leaving unmarked all the constructs of the flesh.

Definition 5.1 (Marked skeleton)**.** *Let* $\mathtt{V}$ *be a set of variables. The marked skeleton of* t *relative to* $\mathtt{V}$ *is defined as* $\mathtt{mskel}^{\mathtt{V}}(t) := t$ *if* $\mathtt{fv}(t) \cap \mathtt{V} = \emptyset$*, otherwise:*

$$\begin{array}{rcl|rcl} \mathtt{mskel}^{\mathtt{V}}(x) & := & x^\circ & \mathtt{mskel}^{\mathtt{V}}(tu) & := & \mathtt{mskel}^{\mathtt{V}}(m) \circ \mathtt{mskel}^{\mathtt{V}}(n) \\ \mathtt{mskel}^{\mathtt{V}}(\lambda x.t) & := & \lambda^\circ x.\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t) & & & \end{array}$$

The marked skeleton of $\lambda x.t$ *is defined as* $\mathtt{mskel}(\lambda x.t) := \lambda^\circ x.\mathtt{mskel}^{\{x\}}(t)$.

Example: for $v := \lambda x.\lambda y.zzx(yz)$, one has $\mathtt{mskel}(v) = \lambda^\circ x.\lambda^\circ y.(zz) \circ x^\circ \circ (y^\circ \circ z)$.

From the Marked Skeleton to the Skeletal Decomposition. Next, we turn marked skeletons into skeletal decompositions, by extracting unmarked sub-terms and turning them into the flesh, while removing the marks from the skeleton. We shall do this via a *splitting function*, defined below, which operates on (parametrized) marked skeletons. For that, we give a lemma guaranteeing that the unmarked sub-terms of marked skeletons can be recognized from their top constructor, that is in $\mathcal{O}(1)$, thus removing the need to inspect them.

Lemma 5.2 (Being unmarked is downward closed in marked skeletons)**.** *Let* $\mathtt{mskel}^{\mathtt{V}}(t) = M\langle m\rangle$ *with* m *starting unmarked. Then* m *has no marks.* Proof at p. 26

The lemma implies the correctness of the first clause of the following definition, as it ensures that in that case m has no marks.

Definition 5.3 (Splitting)**.** *Let* $m = \mathtt{mskel}^{\mathtt{V}}(t)$ *for some* t *and* $\mathtt{V}$*. The following splitting function separates the skeleton from the flesh and removes all marks:*

$$
\begin{array}{rcll}
\mathtt{split}(m) & := & (z, \langle\cdot\rangle[z{\leftarrow}m]) & \textit{if } m \textit{ starts unmarked, is not a variable, and } z \textit{ is fresh;} \\
\mathtt{split}(x) & := & (x, \langle\cdot\rangle); & \\
\mathtt{split}(x^{\circ}) & := & (x, \langle\cdot\rangle); & \\
\mathtt{split}(\lambda^{\circ}x.n) & := & (\lambda x.u, S) & \textit{with } \mathtt{split}(n) = (u, S); \\
\mathtt{split}(m_1 \circ m_2) & := & (t_1 t_2, S'\langle S\rangle) & \textit{with } \mathtt{split}(m_1) = (t_1, S) \textit{ and } \mathtt{split}(m_2) = (t_2, S').
\end{array}
$$

Example: $\mathtt{split}(\lambda^{\circ}x.\lambda^{\circ}y.(zz) \circ x^{\circ} \circ (y^{\circ} \circ z)) = (\lambda x.\lambda y.wx(yz), \langle\cdot\rangle[w{\leftarrow}zz])$.

We can now prove that the marked notions allow one to retrieve the standard ones.

Proposition 5.4 (Soundness of the marked skeleton)**.** Proof at p. 26

Let $t \in \Lambda$ *and* $\mathtt{V}$ *be a set of variables.*

1. Auxiliary parametrized statement*:* $\mathtt{split}(\mathtt{mskel}^{\mathtt{V}}(t)) = \mathtt{skdec}^{\mathtt{V}}(t)$*;*
2. Soundness*:* $\mathtt{skdec}(\lambda x.t) = \mathtt{split}(\mathtt{mskel}(\lambda x.t))$.

Computing the Marked Skeleton. We shall now define a parallel algorithm for computing the marked skeleton via a rewriting system on marked terms. The definition is in Fig. 5. The algorithm shall be applied to abstractions, say to $\lambda x.t$. The algorithm is described by inserting the *up* symbols $\Uparrow$, denoting where the algorithm is operating, and propagating through the term. At the beginning, there is exactly one $\Uparrow$, as the algorithm starts on $\lambda x.\bar{t}^{\Uparrow}$.

The algorithm has two sets of rules. Propagation rules turn the encountered unmarked constructs into marked ones. The interesting rule is $\to_{\mathtt{pr3}}$ for abstractions, that is also the very first one to be applied on the initial term $\lambda x.\bar{t}^{\Uparrow}$. Its effect is that λx is marked as $\lambda^{\circ}x$ and all the occurrences of x in t are also marked, obtaining $\lambda^{\circ}x.t\{x{\leftarrow}x^{\circ}\}$; this is how we recast the role played by Shivers and Wand's bi-directional edges connecting abstractions and variables. Essentially, propagation rules move $\Uparrow$ upwards but they also add $\Uparrow$ at the bottom of the syntax tree on the variable occurrences associated to marked abstractions.

Absorption rules, instead, remove the $\Uparrow$ symbol when it encounters a marked construct, as it means that the concerned thread of the algorithm is passing where another thread already passed before, thus the concerned thread becomes redundant and can be terminated.

Intuitively, the set of $\Uparrow$-sub-terms is the frontier of where the parallel algorithm is operating. When the algorithm is over, no node is marked with $\Uparrow$.

The algorithm is parallel in that the initial substitution $t\{x{\leftarrow}x^{\circ}\}$ and rule $\mapsto_{\mathtt{pr3}}$ might introduce many occurrences of $\Uparrow$, which can be propagated independently.

As an example, consider $v := \lambda x.\lambda y.zzx(yz)$, for which one has, for instance:

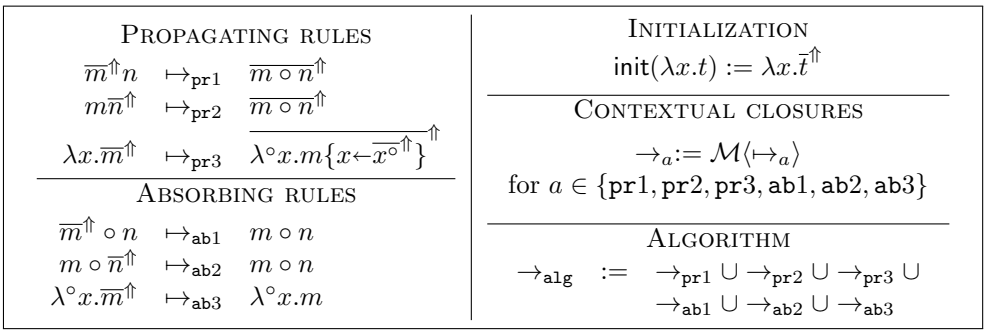


Figure 5: Rewriting rules for marked terms.

$$
\begin{array}{lll}
\mathsf{init}(v) = \lambda x.\overline{\lambda y.zzx(yz)}^{\Uparrow} & \to_{\mathtt{pr3}} & \overline{\lambda^{\circ} x.\lambda y.zz\overline{x^{\circ}}^{\Uparrow}(yz)}^{\Uparrow} \\
\to_{\mathtt{pr2}} \overline{\lambda^{\circ} x.\lambda y.\overline{(zz) \circ x^{\circ}}^{\Uparrow}(yz)}^{\Uparrow} & \to_{\mathtt{pr1}} & \overline{\lambda^{\circ} x.\lambda y.\overline{(zz) \circ x^{\circ} \circ (yz)}^{\Uparrow}}^{\Uparrow} \\
\to_{\mathtt{pr3}} \overline{\lambda^{\circ} x.\overline{\lambda^{\circ} y.(zz) \circ x^{\circ} \circ (\overline{y^{\circ}}^{\Uparrow} z)}^{\Uparrow}}^{\Uparrow} & \to_{\mathtt{pr1}} & \overline{\lambda^{\circ} x.\overline{\lambda^{\circ} y.(zz) \circ x^{\circ} \circ \overline{y^{\circ} \circ z}^{\Uparrow}}^{\Uparrow}}^{\Uparrow} \\
\to_{\mathtt{ab2}} \to_{\mathtt{ab3}} \overline{\lambda^{\circ} x.\overline{\lambda^{\circ} y.(zz) \circ x^{\circ} \circ (y^{\circ} \circ z)}^{\Uparrow}}^{\Uparrow} & = & \overline{\mathtt{mskel}(v)}^{\Uparrow}
\end{array}
$$

Proposition 5.5. *Reduction $\to_{\mathtt{alg}}$ is strongly normalizing and diamond (thus confluent).*

Proof.

1. *Strong normalization.* Let m be a marked term. As terminating measure, consider $(|m|_{\neg\circ}, |m|_{\Uparrow})$ where $|m|_{\neg\circ}$ and $|m|_{\Uparrow}$ are the number of unmarked and $\Uparrow$ constructs in m. Note that the propagation rules decrease $|m|_{\neg\circ}$ while the absorption rules decrease $|m|_{\Uparrow}$ and leave $|m|_{\neg\circ}$ unchanged. Then $\to_{\mathtt{alg}}$ is strongly normalizing.

2. *Diamond.* We prove the diamond property, which implies confluence. Clearly, redexes cannot duplicate or erase each other. There are only two critical pairs, namely:

$$
\begin{array}{ccc}
M\langle \overline{m}^{\Uparrow} \overline{n}^{\Uparrow} \rangle & \xrightarrow{\mathtt{pr1}} & M\langle m \circ \overline{n}^{\Uparrow} \rangle \\
{\scriptstyle \mathtt{pr2}}\downarrow & & \vdots\,{\scriptstyle \mathtt{ab2}} \\
M\langle \overline{m}^{\Uparrow} \circ n \rangle & \overset{\mathtt{ab1}}{\cdots\!\to} & M\langle m \circ n \rangle
\end{array}
\qquad
\begin{array}{ccc}
M\langle \overline{m}^{\Uparrow} \circ \overline{n}^{\Uparrow} \rangle & \xrightarrow{\mathtt{ab1}} & M\langle m \circ \overline{n}^{\Uparrow} \rangle \\
{\scriptstyle \mathtt{ab2}}\downarrow & & \vdots\,{\scriptstyle \mathtt{ab2}} \\
M\langle \overline{m}^{\Uparrow} \circ n \rangle & \overset{\mathtt{ab1}}{\cdots\!\to} & M\langle m \circ n \rangle
\end{array}
\quad \square
$$

To state the correctness of the algorithm and give a bound on the number of its steps, we need two definitions.

Definition 5.6. *Let m be a marked term.*

- Starts with $\circ$/unmarked/$\Uparrow$*: m starts with $\circ$ (resp.* unmarked*, resp.* with $\Uparrow$*) if it has shape x°, $\lambda^{\circ} x.n$, or $n \circ o$ (resp. x, $\lambda x.n$, or no, resp. $\overline{m}^{\Uparrow}$).*
- White size*: the white size $|m|_{\circ}$ of m is the number of $\circ$ constructs in m.*

The following lemma is the key step in the proof of correctness of the algorithm. It is a technical lemma, the statement of which is more easily understood by looking at the proof of the following theorem.

Lemma 5.7 (Crucial). *Let t be a term, $\mathtt{V}$ a set of variables, $x \in \mathtt{fv}(t) \setminus \mathtt{V}$, $m := \mathtt{mskel}^{\mathtt{V}}(t)$, and $n := \mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t)$. Then:*

Proof at p. 27

$$m\{x\leftarrow\overline{x^{\circ}}^{\Uparrow}\} \to_{\mathtt{alg}}^{k} \begin{cases} \overline{n}^{\Uparrow} & \text{with } k = |n|_{\circ} - |m|_{\circ} - 1, \text{ if } m \text{ starts unmarked;} \\ n & \text{with } k = |n|_{\circ} - |m|_{\circ}, \text{ if } m \text{ starts with } \circ. \end{cases}$$

Theorem 5.8 (Soundness of the algorithm). *Let t be a term. Then $\mathsf{init}(\lambda x.t) \to_{\mathtt{alg}}^{n} \overline{\mathtt{mskel}(\lambda x.t)}^{\Uparrow}$ with $n = |\mathtt{mskel}(\lambda x.t)|_{\circ}$.*

Proof. By definition $\mathsf{init}(\lambda x.t) = \lambda x.\overline{t}^{\Uparrow} \to_{\mathtt{pr3}} \overline{\lambda^{\circ}x.t\{x\leftarrow\overline{x^{\circ}}^{\Uparrow}\}}^{\Uparrow} = \overline{\lambda^{\circ}x.\mathtt{mskel}^{\emptyset}(t)\{x\leftarrow\overline{x^{\circ}}^{\Uparrow}\}}^{\Uparrow}$, while $\mathtt{mskel}(\lambda x.t) = \lambda^{\circ}x.\mathtt{mskel}^{\{x\}}(t)$. We consider two cases.

- **Case** $x \notin \mathtt{fv}(t)$. Then $\mathtt{mskel}^{\{x\}}(t) = t$ and $t\{x\leftarrow\overline{x^{\circ}}^{\Uparrow}\} = t$, so $\lambda^{\circ}x.t\{x\leftarrow\overline{x^{\circ}}^{\Uparrow}\} = \lambda^{\circ}x.t = \lambda^{\circ}x.\mathtt{mskel}^{\{x\}}(t) = \mathtt{mskel}(\lambda x.t)$. Thus, $\mathsf{init}(\lambda x.t) \to_{\mathtt{pr3}} \overline{\mathtt{mskel}(\lambda x.t)}^{\Uparrow}$. Moreover, $n = 1 = |\mathtt{mskel}(\lambda x.t)|_{\circ}$, since the only white construct of $\mathtt{mskel}(\lambda x.t)$ is the root abstraction.

- **Case** $x \in \mathtt{fv}(t)$. By Lemma 5.7, $\lambda^{\circ}x.t\{x\leftarrow\overline{x^{\circ}}^{\Uparrow}\} \to_{\mathtt{alg}}^{k} \lambda^{\circ}x.\overline{\mathtt{mskel}^{\{x\}}(t)}^{\Uparrow}$, where $k = |\mathtt{mskel}^{\{x\}}(t)|_{\circ} - |t|_{\circ} - 1 = |\mathtt{mskel}^{\{x\}}(t)|_{\circ} - 1$. Therefore:

$$\begin{array}{rlcl} \mathsf{init}(\lambda x.t) & \to_{\mathtt{pr3}} & \overline{\lambda^{\circ}x.t\{x\leftarrow\overline{x^{\circ}}^{\Uparrow}\}}^{\Uparrow} & \to_{\mathtt{alg}}^{k} \quad \overline{\lambda^{\circ}x.\overline{\mathtt{mskel}^{\{x\}}(t)}^{\Uparrow}}^{\Uparrow} \\ & \to_{\mathtt{ab3}} & \overline{\lambda^{\circ}x.\mathtt{mskel}^{\{x\}}(t)}^{\Uparrow} & = \quad \overline{\mathtt{mskel}(\lambda x.t)}^{\Uparrow}. \end{array}$$

  Now, $n = k+2 = |\mathtt{mskel}^{\{x\}}(t)|_{\circ} - 1 + 2 = |\mathtt{mskel}^{\{x\}}(t)|_{\circ} + 1 = |\mathtt{mskel}(\lambda x.t)|_{\circ}$. □

5.1 Complexity Analysis

We proved that the number of rewriting steps of the algorithm for computing the marked skeleton $\mathtt{mskel}(v)$ is exactly its marked size $|\mathtt{mskel}(\lambda x.t)|_{\circ}$ (Theorem 5.8), which is the size of the skeleton (Prop. 5.4). The complexity of the algorithm is indeed linear but it requires some discussion, since $\to_{\mathtt{pr3}}$ steps are not constant time operations. Moreover, we need to take into account the cost of splitting.

Complexity Analysis 1: Propagation + Absorption. For computing $\mathtt{mskel}(v)$ from v, we assume that abstractions have pointers to the occurrences of the variable, so that the substitution $m\{x\leftarrow\overline{x^{\circ}}^{\Uparrow}\}$ in $\to_{\mathtt{pr3}}$ does not require going through the whole of m. This is Shivers and Wand's key idea, and also how our implementation works (see Appendix G). A global argument gives the cost: the algorithm turns $|\mathtt{mskel}(v)|_{\circ}$ unmarked constructs into $\circ$-constructs exactly once, and it generates at most $|\mathtt{mskel}(v)|_{\circ}$ $\Uparrow$-constructs that are never duplicated and are absorbed exactly once, thus it globally requires $\mathcal{O}(|\mathtt{mskel}(v)|_{\circ})$ time.

Lemma 5.9 (Cost of propagation + absorption). *Computing $\mathtt{mskel}(v)$ from v requires time $\mathcal{O}(|\mathtt{mskel}(v)|_{\circ})$.*

Complexity Analysis 2: Splitting. For the cost of splitting, Lemma 5.2 guarantees that the unmarked sub-terms of marked skeletons can be recognized in $\mathcal{O}(1)$ by only inspecting their topmost constructor, so that implementing the splitting function takes time proportional to the number of marked constructors only.

Lemma 5.10 (Cost of splitting)**.** *Computing* $\mathtt{split}(\mathtt{mskel}(v))$ *from* $\mathtt{mskel}(v)$ *requires time* $\mathcal{O}(|\mathtt{mskel}(v)|_\circ)$.

The next theorem sums it all up, recasting the obtained costs in the unmarked setting via the soundness of the algorithm (Prop. 5.4).

Theorem 5.11 (The algorithm is linear in the skeleton)**.** *Let* $v \in \Lambda$ *be a value. Then the skeletal decomposition* $(\mathtt{skel}(v), \mathtt{flesh}(v))$ *of* v *can be computed in* $\mathcal{O}(|\mathtt{skel}(v)|)$.

6 Preliminaries about Abstract Machines

In this section, we introduce terminology and basic concepts about abstract machines, that shall be the topic of the following sections.

Abstract Machines Glossary. Abstract machines manipulate *pre-terms*, that is, terms without implicit α-renaming. In this paper, an *abstract machine* is a quadruple $\mathtt{M} = (\mathtt{States}, \rightsquigarrow, \cdot \triangleleft \cdot, \underline{\cdot})$ the components of which are as follows.

- *States.* A state $\mathtt{Q} \in \mathtt{States}$ is a quadruple $(\mathtt{C}, t, \mathtt{S}, \mathtt{E})$ composed by the *active term* t, plus three data structure, namely the *chain* $\mathtt{C}$, the *(applicative) stack* $\mathtt{S}$, and the *(global) environment* $\mathtt{E}$. Terms in states are actually pre-terms.
- *Transitions.* The pair $(\mathtt{States}, \rightsquigarrow)$ is a transition system with transitions $\rightsquigarrow$ partitioned into *principal transitions*, whose union is noted $\rightsquigarrow_{\mathtt{pr}}$ and that are meant to correspond to steps on the calculus, and *search transitions*, whose union is noted $\rightsquigarrow_{\mathtt{sea}}$, that take care of searching for (principal) redexes.
- *Initialization.* The component $\triangleleft \subseteq \Lambda \times \mathtt{States}$ is the *initialization relation* associating closed terms without ESs to initial states. It is a *relation* and not a function because $t \triangleleft \mathtt{Q}$ maps a *closed* λ-term t (considered modulo α) to a state $\mathtt{Q}$ having a *pre-term representant* of t (which is not modulo α) as active term. Intuitively, any two states $\mathtt{Q}$ and $\mathtt{Q}'$ such that $t \triangleleft \mathtt{Q}$ and $t \triangleleft \mathtt{Q}'$ are α-equivalent. A state $\mathtt{Q}$ is *reachable* if it can be reached starting from an initial state, that is, if $\mathtt{Q}' \rightsquigarrow^* \mathtt{Q}$ where $t \triangleleft \mathtt{Q}'$ for some t and $\mathtt{Q}'$, shortened as $t \triangleleft \mathtt{Q}' \rightsquigarrow^* \mathtt{Q}$.
- *Read-back.* The read-back function $\underline{\cdot} : \mathtt{States} \to \mathtt{Terms}$ turns reachable states into terms (possibly with ESs, that is, $\mathtt{Terms} = \Lambda_{\mathtt{es}}$ or $\mathtt{Terms} = \Lambda_{\mathtt{sk}}$) and satisfies the *initialization constraint*: if $t \triangleleft \mathtt{Q}$ then $\underline{\mathtt{Q}} =_\alpha t$.

A state is *final* if no transitions apply. A *run* $r : \mathtt{Q} \rightsquigarrow^* \mathtt{Q}'$ is a possibly empty finite sequence of transitions, the length of which is noted $|r|$; note that the first and the last states of a run are not necessarily initial and final. If a and b are

transitions labels (that is, $\rightsquigarrow_a \subseteq \rightsquigarrow$ and $\rightsquigarrow_b \subseteq \rightsquigarrow$) then $\rightsquigarrow_{a,b} := \rightsquigarrow_a \cup \rightsquigarrow_b$ and $|r|_a$ is the number of a transitions in r, $|r|_{a,b} := |r|_a + |r|_b$, and similarly for more than two labels.

For the machines at work in this paper, the pre-terms in initial states shall be *well-bound*, that is, they have pairwise distinct bound names; for instance $(\lambda x.x)\lambda y.y$ is well-bound while $(\lambda x.x)\lambda x.x$ is not. We shall also write t^α in a state $\mathtt{Q}$ for a *fresh well-bound renaming* of t, *i.e.* t^α is α-equivalent to t, well-bound, and its bound variables are fresh with respect to those in t and in the other components of $\mathtt{Q}$.

Mechanical Bismulations. Machines are usually showed to be correct with respect to a strategy via some form of bisimulation relating terms and machine states. The notion that we adopt is here dubbed *mechanical bisimulation*, and it should not be confused with the strong bisimulation property of structural equivalence (which actually plays a role in mechanical bisimulations). The definition, tuned towards complexity analyses, requires a perfect match between the steps of the evaluation sequence and the principal transitions of the machine run. *Terminology*: a structural strategy $(\rightarrow_{\mathtt{str}}, \equiv)$ is a strategy $\rightarrow_{\mathtt{str}}$ plus a relation $\equiv$ that is a strong bisimulation with respect to $\rightarrow_{\mathtt{str}}$ (see Prop. 3.4).

Definition 6.1 (Mechanical bisimulation)**.** *A machine* $\mathtt{M} = (\mathtt{States}, \rightsquigarrow, \cdot \triangleleft \cdot, \underline{\cdot})$ *and a structural strategy* $(\rightarrow_{\mathtt{str}}, \equiv)$ *on* $\Lambda_{\mathtt{sk}}$*-terms are mechanical bisimilar when, given an initial state* $t \triangleleft \mathtt{Q}$*:*

1. Runs to evaluations*: for any run* $r : t \triangleleft \mathtt{Q} \rightsquigarrow^* \mathtt{Q}'$ *there exists an evaluation* $\mathtt{e} : t \rightarrow^*_{\mathtt{str}} \equiv \underline{\mathtt{Q}'}$*;*
2. Evaluations to runs*: for every evaluation* $\mathtt{e} : t \rightarrow^*_{\mathtt{str}} u$ *there exists a run* $r : t \triangleleft \mathtt{Q} \rightsquigarrow^* \mathtt{Q}'$ *such that* $\underline{\mathtt{Q}'} \equiv u$*;*
3. Principal matching*: for every principal transition* $\rightsquigarrow_{\mathtt{a}}$ *of label* a *of* $\mathtt{M}$*, in both previous points the number* $|r|_{\mathtt{a}}$ *of* $\mathtt{a}$*-transitions in* r *is exactly the number* $|\mathtt{e}|_a$ *of* $\mathtt{a}$*-steps in the evaluation* $\mathtt{e}$*,* i.e. $|\mathtt{e}|_{\mathtt{a}} = |r|_{\mathtt{a}}$.

The proof that a machine and a strategy are in a mechanical bisimulation follows from some basic properties, grouped under the notion of distillery, following Accattoli et al. [4].

Definition 6.2 (Distillery)**.** *A machine* $\mathtt{M} = (\mathtt{States}, \rightsquigarrow, \cdot \triangleleft \cdot, \underline{\cdot})$ *and a structural strategy* $(\rightarrow_{\mathtt{str}}, \equiv)$ *are a* distillery *if the following conditions hold:*

1. Principal projection*:* $\mathtt{Q} \rightsquigarrow_{\mathtt{a}} \mathtt{Q}'$ *implies* $\underline{\mathtt{Q}} \rightarrow_{\mathtt{a}} \equiv \underline{\mathtt{Q}'}$ *for every principal transition of label* $\mathtt{a}$*;*
2. Search transparency*:* $\mathtt{Q} \rightsquigarrow_{\mathtt{sea}} \mathtt{Q}'$ *implies* $\underline{\mathtt{Q}} = \underline{\mathtt{Q}'}$*;*
3. Search transitions terminate*:* $\rightsquigarrow_{\mathtt{sea}}$ *terminates;*
4. Determinism*:* $\rightarrow_{\mathtt{str}}$ *is deterministic;*
5. Halt*:* $\mathtt{M}$ *final states decode to* $\rightarrow_{\mathtt{str}}$*-normal terms.*

Theorem 6.3 (Sufficient condition for implementations)**.** *Let a machine* $\mathtt{M}$ *and a structural strategy* $(\rightarrow_{\mathtt{str}}, \equiv)$ *be a distillery. Then, they are mechanical bisimilar.* Proof at p. 32

STACKS	$\mathtt{S}$	$::=$	$\epsilon \mid t{:}\mathtt{S}$	CHAINS	$\mathtt{C}$	$::=$	$\epsilon \mid \mathtt{C}{:}(x, \mathtt{S}, \mathtt{E}[x{\leftarrow}\cdot])$
MAD ENVS	$\mathtt{E}$	$::=$	$\epsilon \mid [x{\leftarrow}t]{:}\mathtt{E}$	STATES	$\mathtt{Q}$	$:=$	$(\mathtt{C}, t, \mathtt{S}, \mathtt{E})$
SKEL. MAD ENVS	$\mathtt{E}$	$::=$	$\epsilon \mid [x{\leftarrow}t]{:}\mathtt{E} \mid \langle x{\leftarrow}v\rangle{:}\mathtt{E}$	INIT.	t	$\triangleleft$	$(\epsilon, t^{\alpha}, \epsilon, \epsilon)$ (#)

COMMON TRANSITIONS

Chain	Code	Stack	Env		Chain	Code	Stack	Env	
$\mathtt{C}$	tu	$\mathtt{S}$	$\mathtt{E}$	$\rightsquigarrow_{\mathtt{sea}_1}$	$\mathtt{C}$	t	$u{:}\mathtt{S}$	$\mathtt{E}$	
$\mathtt{C}$	$\lambda x.t$	$u{:}\mathtt{S}$	$\mathtt{E}$	$\rightsquigarrow_{\beta}$	$\mathtt{C}$	t	$\mathtt{S}$	$[x{\leftarrow}u]{:}\mathtt{E}$	
$\mathtt{C}$	x	$\mathtt{S}$	$\mathtt{E}{:}[x{\leftarrow}t]{:}\mathtt{E}'$	$\rightsquigarrow_{\mathtt{sea}_2}$	$\mathtt{C}{:}(x, \mathtt{S}, \mathtt{E}[x{\leftarrow}\cdot])$	t	ϵ	$\mathtt{E}'$	$(*)$
$\mathtt{C}{:}(x, \mathtt{S}, \mathtt{E}[x{\leftarrow}\cdot])$	v	ϵ	$\mathtt{E}'$	$\rightsquigarrow_{\mathtt{sea}_3}$	$\mathtt{C}$	x	$\mathtt{S}$	$\mathtt{E}{:}[x{\leftarrow}v]{:}\mathtt{E}'$	

MAD SPECIFIC TRANSITION

$\mathtt{C}$	x	$\mathtt{S}$	$\mathtt{E}{:}[x{\leftarrow}v]{:}\mathtt{E}'$	$\rightsquigarrow_{\mathtt{sub}}$	$\mathtt{C}$	v^{α}	$\mathtt{S}$	$\mathtt{E}{:}[x{\leftarrow}v]{:}\mathtt{E}'$	(#)

SKELETAL MAD SPECIFIC TRANSITIONS

$\mathtt{C}$	x	$\mathtt{S}$	$\mathtt{E}{:}[x{\leftarrow}v]{:}\mathtt{E}'$	$\rightsquigarrow_{\mathtt{sk}}$	$\mathtt{C}$	x	$\mathtt{S}$	$\mathtt{E}{:}\langle x{\leftarrow}v'\rangle{:}\mathtt{E}_S{:}\mathtt{E}'$	(%)
$\mathtt{C}$	x	$\mathtt{S}$	$\mathtt{E}{:}\langle x{\leftarrow}v\rangle{:}\mathtt{E}'$	$\rightsquigarrow_{\mathtt{ss}}$	$\mathtt{C}$	v^{α}	$\mathtt{S}$	$\mathtt{E}{:}\langle x{\leftarrow}v\rangle{:}\mathtt{E}'$	(#)

(#) t^{α} is any well-bound code α-equivalent to t such that its bound names are fresh with respect to those in the rest of the state.
(*) If t is not a value (%) Where $\mathtt{skdec}(v) = (v', S)$ is the skeletal decomposition of v (computed as $\mathtt{split}(\mathtt{mskel}(v))$), and $\mathtt{E}_S$ is S seen as an environment.

READ-BACK

EMPTY		$\underline{\epsilon}$	$:=$	$\langle\cdot\rangle$	ENVS	$\underline{[x{\leftarrow}t]{:}\mathtt{E}}$	$:=$	$\underline{\mathtt{E}}\langle\langle\cdot\rangle[x{\leftarrow}t]\rangle$
STACKS		$\underline{t{:}\mathtt{S}}$	$:=$	$\underline{\mathtt{S}}\langle\langle\cdot\rangle t\rangle$		$\underline{\langle x{\leftarrow}t\rangle{:}\mathtt{E}}$	$:=$	$\underline{\mathtt{E}}\langle\langle\cdot\rangle\langle x{\leftarrow}t\rangle\rangle$
CHAINS	$\underline{\mathtt{C}{:}(x, \mathtt{S}, \mathtt{E}[x{\leftarrow}\cdot])}$		$:=$	$\underline{\mathtt{E}}\langle\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\langle x\rangle\rangle\rangle[x{\leftarrow}\langle\cdot\rangle]$	STATES	$\underline{(\mathtt{C}, t, \mathtt{S}, \mathtt{E})}$	$:=$	$\underline{\mathtt{E}}\langle\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\langle t\rangle\rangle\rangle$

Figure 6: The Milner Abstract machine by Need (MAD) and the Skeletal Milner Abstract machine by Need (Skeletal MAD) presented as different extension of a common set of transitions (but be careful: the common transitions use different notions of environment).

7 The MAD and the Skeletal MAD

In this section, we study an abstract machine for skeletal CbNeed, obtained as a minor modification of a known machine for CbNeed, that is recalled here. Since the two machines share most data structures and transitions, they are presented together in Fig. 6 (slightly abusing notations: the two machines define environments $\mathtt{E}$ differently, so the notation of the common transitions is overloaded).

The MAD. The *Milner Abstract machine by neeD* (MAD) of Fig. 6 implements the CbNeed strategy $\rightarrow_{\mathtt{need}}$. It is a CbNeed variant of the *Milner Abstract Machine* (MAM), itself a simplification of the Krivine abstract machine (KAM). The MAM is a CbN abstract machine using a single global environment (sometimes called *heap*), a stack for arguments, and no closures—it is omitted here (the KAM instead uses closures and many local environments). Both the MAM and the MAD are introduced by Accattoli et al. in [4].

The MAD modifies the MAM by adding a mechanism realizing the memoization characteristic of CbNeed. Namely, when execution encounters a variable occurrence x associated to an environment entry $[x{\leftarrow}u]$ such that u is not a value, then the MAM duplicates u while the MAD does not. Instead, part of the current state is saved on the new *chain* data structure $\mathtt{C}$, and the MAD starts evaluating u inside the environment entry itself; this is transition $\rightsquigarrow_{\mathtt{sea}_2}$.

When such an evaluations ends with a value v, the machines resumes the state saved on the chain, and the environment is updated replacing u with v—this is transition $\rightsquigarrow_{\mathtt{sea}_3}$—so that future encounters of x will not need to re-evaluate u. Both the MAM and the MAD rely on α-renaming as a black-box operation $\cdot^{\alpha}$, at work in transition $\rightsquigarrow_{\mathtt{sub}}$.

The presentation in Fig. 6 differs from the MAD in [4] in a very minor point. In [4], transitions $\rightsquigarrow_{\mathtt{sea}_3}$ and $\rightsquigarrow_{\mathtt{sub}}$ are concatened into a single transition. Splitting them helps in stating the invariants of the machines.

The Skeletal MAD. The Skeletal MAD, also in Fig. 6, modifies the MAD exactly as the Skeletal CbNeed LSC modifies the CbNeed LSC, that is, by adding skeletal entries $\langle x \leftarrow v \rangle$ to the global environment and splitting the substitution transtition $\rightsquigarrow_{\mathtt{sub}}$ into two: the skeletonizing transition $\rightsquigarrow_{\mathtt{sk}}$ that separates the skeleton v' from the flesh of the value v, and the skeletal substitution transition $\rightsquigarrow_{\mathtt{ss}}$ that substitutes the (α-renamed) skeleton v'^{α}.

Notation. For specifying transition $\rightsquigarrow_{\mathtt{sk}}$, we use the following notation for the flesh: a substitution context $S = \langle\cdot\rangle[\![x_1 \leftarrow t_1]\!] \dots [\![x_k \leftarrow t_k]\!]$, where each $[\![x_i \leftarrow t_i]\!]$ can be $[x_i \leftarrow t_i]$ or $\langle x_i \leftarrow t_i \rangle$, induces a global environment $\mathtt{E}_S := [\![x_1 \leftarrow t_1]\!] : \dots : [\![x_k \leftarrow t_k]\!] : \epsilon$.

The union of all the transition rules of the Skeletal MAD (namely, $\mathtt{sea}_1$, $\mathtt{sea}_2$, $\mathtt{sea}_3$, β, $\mathtt{sk}$, and $\mathtt{ss}$) is noted $\rightsquigarrow_{\mathtt{SMAD}}$.

The read-back of (Skeletal) MAD states to terms (possibly with ESs) is defined in Fig. 6. The definition uses auxiliary notions of read-back for chains, stacks, and environments that turn them into CbNeed evaluation contexts. For stacks and environments, their read-back is clearly an evaluation context. For chains, it shall be proved as an invariant of the machine.

Invariants. To prove properties of the Skeletal MAD we need to isolate some of its invariants in Lemma 7.2 below. The closure one ensures that the closure of the initial term extends, in an appropriate sense, to all reachable states. The well-bound invariant, similarly, ensures that binders in reachable states are pairwise distinct, as in the initial term. To compactly formulate the closure invariant we need the notion of terms of a state.

Definition 7.1 (Terms of a state)**.** *Let* $\mathtt{Q} = (\mathtt{C}, u, \mathtt{S}, \mathtt{E})$ *be a Skeletal MAD state where* $\mathtt{C}$ *is a possibly empty chain* $\epsilon : (x_k, \mathtt{S}_k, \mathtt{E}_k : [x_k \leftarrow \cdot]) : \dots : (x_1, \mathtt{S}_1, \mathtt{E}_1 : [x_1 \leftarrow \cdot])$ *for some* $k \geq 0$*. The* terms of $\mathtt{Q}$ *are* u*, every term in* $\mathtt{S}$ *and* $\mathtt{S}_i$*, and every term in an entry of* $\mathtt{E}$ *or* $\mathtt{E}_i$*, for* $1 \leq i \leq k$*.*

Lemma 7.2 (Qualitative invariants)**.** *Let* $t \triangleleft \mathtt{Q} \rightsquigarrow^* \mathtt{Q}' = (\mathtt{C}, u, \mathtt{S}, \mathtt{E})$ *be a Skeletal MAD run.* Proof at p. 33

1. Closure*: for every term* s *of* $\mathtt{Q}'$ *and for any variable* $x \in \mathtt{fv}(s)$ *there is an environment entry* $[x \leftarrow t]$*,* $\langle x \leftarrow v \rangle$*, or* $[x \leftarrow \cdot]$ *on the right of* s *in* $\mathtt{Q}'$*.*

2. Well-bound*: if* $\lambda x.s$ *occurs in* $\mathtt{Q}'$ *and* x *has any other occurrence in* $\mathtt{Q}'$ *then it is as a free variable of* s*, and for any substitution* $[y \leftarrow t]$*,* $\langle y \leftarrow v \rangle$*, or* $[y \leftarrow \cdot]$ *in* $\mathtt{Q}'$ *the name* y *can occur (in any form) only on the left of that entry in* $\mathtt{Q}'$*.*

3. Contextual chain read-back*:* $\underline{\mathtt{C}}$*,* $\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\rangle$*,* $\underline{\mathtt{E}}\langle\underline{\mathtt{C}}\rangle$ *and* $\underline{\mathtt{E}}\langle\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\rangle\rangle$ *are CbNeed evaluation contexts.*

Mechanical Bisimulation. The invariants allow us to prove that the Skeletal MAD and the strategy $\to_{\mathtt{sk{\cdot}need}}$ modulo $\equiv$ form a distillery, from which the mechanical bisimulation follows (by Theorem 6.3). The closure invariant is used for Point 4, the other two for Point 1.

Theorem 7.3 (Skeletal distillery)**.** *The Skeletal MAD and the structural strategy* $(\to_{\mathtt{sk{\cdot}need}}, \equiv)$ *form a distillery. Namely, let* $\mathtt{Q}$ *be a reachable Skeletal MAD state.* Proof at p. 34

1. Principal projection*:*
   (a) *if* $\mathtt{Q} \rightsquigarrow_{\beta} \mathtt{Q}'$ *then* $\underline{\mathtt{Q}} \to_{\mathtt{sk{\cdot}dB}} \equiv \underline{\mathtt{Q}'}$.
   (b) *if* $\mathtt{Q} \rightsquigarrow_{\mathtt{sk}} \mathtt{Q}'$ *then* $\underline{\mathtt{Q}} \to_{\mathtt{sk}} \underline{\mathtt{Q}'}$.
   (c) *if* $\mathtt{Q} \rightsquigarrow_{\mathtt{ss}} \mathtt{Q}'$ *then* $\underline{\mathtt{Q}} \to_{\mathtt{ss}} \underline{\mathtt{Q}'}$.
2. Search transparency*: if* $\mathtt{Q} \rightsquigarrow_{\mathtt{sea}_1,\mathtt{sea}_2,\mathtt{sea}_3} \mathtt{Q}'$ *then* $\underline{\mathtt{Q}} = \underline{\mathtt{Q}'}$.
3. Search terminates*:* $\rightsquigarrow_{\mathtt{sea}_1,\mathtt{sea}_2,\mathtt{sea}_3}$ *is terminating.*
4. Halt*: if* $\mathtt{Q}$ *is final then it has shape* $(\epsilon, v, \epsilon, \mathtt{E})$ *and* $\underline{\mathtt{Q}}$ *is* $\to_{\mathtt{sk{\cdot}need}}$*-normal.*

Corollary 7.4 (Skeletal mechanical bisimulation)**.** *The Skeletal MAD and the structural strategy* $(\to_{\mathtt{sk{\cdot}need}}, \equiv)$ *are in a mechanical bisimulation.*

8 Complexity Analysis

In this section, we show that the Skeletal MAD can be concretely implemented within the same complexity of the MAD, that is, with *bi-linear* overhead.

Sub-Term Property. As it is standard, the complexity analysis crucially relies on the sub-term property, that bounds the size of manipulated (and thus duplicated) terms using the size of the initial term. The proof requires a similar property about skeletons.

Lemma 8.1 (Skeleton and flesh size)**.** Proof at p. 35

1. Auxiliary parametrized statement*: let* $t \in \Lambda$*,* $\mathtt{V}$ *be a set of variables, and* $\mathtt{skdec}^{\mathtt{V}}(t) := (u, S)$*. Then* $|u| \leq |t|$ *and* $|s| \leq |t|$ *for any ES* $[x \leftarrow s]$ *in* S*;*
2. Main*: let* $v \in \Lambda$*. Then* $|\mathtt{skel}(v)| \leq |v|$ *and* $|s| \leq |v|$ *for any ES* $[x \leftarrow s]$ *in* $\mathtt{flesh}(v)$*.*

Lemma 8.2 (Quantitative sub-term property)**.** *Let* $t \triangleleft \mathtt{Q} \rightsquigarrow^{*}_{\mathtt{SMAD}} \mathtt{Q}'$ *be a Skeletal MAD run. Then* $|u| \leq |t|$ *for any term* u *of* $\mathtt{Q}'$*.* Proof at p. 35

Next, some basic observations about the transitions, together with the sub-term property, allow us to bound their number using the two key parameters (that is, number of β-steps/transitions and size of the initial term).

Proposition 8.3 (Number of transitions)**.** *Let* $r : t \triangleleft \mathtt{Q} \rightsquigarrow^{*}_{\mathtt{SMAD}} \mathtt{Q}'$ *be a Skeletal MAD run.*

1. $|r|_{\mathtt{sk},\mathtt{ss},\mathtt{sea}_2,\mathtt{sea}_3} \in \mathcal{O}(|r|_{\beta})$*, and*

2. $|r|_{\mathtt{sea}_1} \in \mathcal{O}(|t| \cdot (|r|_\beta + 1))$.

Proof.

1. (a) $|r|_{\mathtt{sea}_2} \leq |r|_\beta$, because every $\mathtt{sea}_2$ transition consumes one non-value entry from the environment, which are created only by β transitions.
   (b) $|r|_{\mathtt{sea}_3} \leq |r|_\beta$, because every $\mathtt{sea}_3$ transition consumes one entry from the chain, which are created only by $\mathtt{sea}_2$ transitions, which in turn are bound by β transitions.
   (c) $|r|_{\mathtt{ss}} \in \mathcal{O}(|r|_\beta)$, because after a $\mathtt{ss}$ transition there can be either a $\mathtt{sea}_3$ transition, a β transition, or no transition (if the $\mathtt{ss}$ transition is the last one of the run r). Therefore, $|r|_{\mathtt{ss}} \leq |r|_{\mathtt{sea}_3} + |r|_\beta + 1 \leq 2|r|_\beta + 1$.
   (d) $|r|_{\mathtt{sk}} \in \mathcal{O}(|r|_\beta)$, because every $\mathtt{sk}$ transition is followed by a $\mathtt{ss}$ transition.
2. Note that $\mathtt{sea}_1$ transitions decrease the size of the code, which is increased only by transitions $\mathtt{ss}$ and $\mathtt{sea}_2$. By the sub-term property (Lemma 8.2), the code increase is bounded by the size $|t|$ of the initial term. By Point 1, $|r|_{\mathtt{ss},\mathtt{sea}_2} = \mathcal{O}(|r|_\beta)$. Then $|r|_{\mathtt{sea}_1} \in \mathcal{O}(|t| \cdot (|r|_{\mathtt{ss},\mathtt{sea}_2} + 1)) = \mathcal{O}(|t| \cdot (|r|_\beta + 1))$. □

Lastly, we need some assumptions on how the Skeletal MAD can be concretely implemented. Our OCaml implementation (see Appendix G) represents variables as memory locations and variable occurrences as pointers to those locations, obtaining random access to environment entries in $\mathcal{O}(1)$. As already mentioned in Sect. 5, all constructors have pointers to their parents in the syntactic tree, and in particular variables have pointers to their occurrences, as to implement efficiently the computation of the skeleton. Moreover, environments are implemented as doubly linked lists, to split/concatenate them in $\mathcal{O}(1)$ in transitions $\mathtt{sea}_2$ and $\mathtt{sea}_3$. The cost of transitions $\mathtt{sk}$ and $\mathtt{ss}$ is bound by the size of the value to skeletonize/copy, which is bound by the sub-term property.

Lemma 8.4 (Cost of single transitions)**.** *Let* $t \triangleleft \mathtt{Q} \rightsquigarrow^*_{\mathtt{SMAD}} \mathtt{Q}'$ *be a run. Implementing any transitions from* $\mathtt{Q}'$ *costs* $\mathcal{O}(1)$ *but for* $\mathtt{sk}$ *and* $\mathtt{ss}$ *transitions, which cost* $\mathcal{O}(|t|)$ *each.*

Putting all together, we obtain a bilinear bound.

Theorem 8.5 (The Skeletal MAD is bi-linear)**.** *Let* $r : t \triangleleft \mathtt{Q} \rightsquigarrow^*_{\mathtt{SMAD}} \mathtt{Q}'$ *be a Skeletal MAD run. Then* r *can be implemented on random access machines in* $\mathcal{O}(|t| \cdot (|r|_\beta + 1))$.

9 Conclusions

We show that skeletal call-by-need provides, in some cases, exponentially shorter evaluation sequences involving exponentially smaller terms than call-by-need. We also show that the required skeleton reconstruction can be implemented in linear time and space, by giving a new simpler and graph-free presentation of ideas by Shivers and Wand. Lastly, we smoothly plug this result in the existing distillation technique for abstract machines, obtaining an implementation of

skeletal call-by-need with bi-linear overhead. In particular, the bi-linear overhead allows us to establish that the shrinking of evaluation sequences is a real time speed-up: the smart specification of skeletal call-by-need does not hide an expensive overhead.

A Proofs Omitted from Sect. 3 (Skeletal Call-by-Need)

Proof of Determinism. We first need an auxiliary result lemma.

Lemma A.1. *Let $t := E\langle\!\langle x \rangle\!\rangle$ for an evaluation context E. Then:*

1. Not an answer*: $t \neq S\langle v \rangle$ for every substitution context S and abstraction v;*
2. Unique decomposition*: for every evaluation context E' and variable y, $t = E'\langle\!\langle y \rangle\!\rangle$ implies $E' = E$ and $y = x$;*
3. Normal*: t is a $\to_{\mathtt{sk\cdot need}}$ normal form.*

Proof. In all points we use a structural induction on E.

1. *Not an answer*:
   - $E = \langle\cdot\rangle$: obvious.
   - $E = E_1 u$: obvious.
   - $E = E_1[y{\leftarrow}s]$: suppose that $S = S'[y{\leftarrow}s]$ (for otherwise the result is obvious); then we apply the *i.h.* to E_1 to obtain $E_1\langle x \rangle \neq S'\langle v \rangle$.
   - $E = E_1\langle y{\leftarrow}v \rangle$: analogous to the previous case.
   - $E = E_1\langle\!\langle y \rangle\!\rangle[y{\leftarrow}E_2]$: suppose that $S = S'[y{\leftarrow}E_2\langle x \rangle]$ (for otherwise the result is obvious); then we apply the *i.h.* to E_1 to obtain $E_1\langle\!\langle y \rangle\!\rangle \neq S'\langle v \rangle$.
2. *Unique decomposition*:
   - $E = \langle\cdot\rangle$: obvious.
   - $E = E_1 u$: then necessarily $E' = E'_1 u$, and it follows from the *i.h.*
   - $E = E_1\langle z{\leftarrow}u \rangle$: then necessarily $E' = E'_1\langle z{\leftarrow}u \rangle$, and it follows from the *i.h.*
   - $E = E_1[z{\leftarrow}u]$: two sub-cases.
     (a) $E' = E'_1[z{\leftarrow}u]$: it follows from the *i.h.*, as above.
     (b) $E' = E'_1\langle\!\langle z \rangle\!\rangle[z{\leftarrow}E'_2]$, with $E'_2\langle\!\langle y \rangle\!\rangle = u$. This is actually impossible. In fact, it implies $E_1\langle\!\langle x \rangle\!\rangle = E'_1\langle\!\langle z \rangle\!\rangle$, which by *i.h.* gives us $z = x$, contradicting the hypothesis that E does not capture x.
   - $E = E_1\langle\!\langle z \rangle\!\rangle[z{\leftarrow}E_2]$: by symmetry with the above case, the only possibility is $E' = E_1\langle\!\langle z \rangle\!\rangle[z{\leftarrow}E'_2]$, that follows from the *i.h.*
3. *Normal*: let r be a redex (*i.e.*, a term matching the left-hand side of $\mapsto_{\mathtt{dB}}$, $\mapsto_{\mathtt{sk}}$, or $\mapsto_{\mathtt{ss}}$) and let E' be an evaluation context. We shall show by structural induction on E that $t \neq E'\langle r \rangle$. We shall do this by considering, in each inductive case, all the possible shapes of E'.

- $E = \langle\cdot\rangle$: obvious.
- $E = E_1 u$: two sub-cases.
  (a) $E' = \langle\cdot\rangle$. Since r is a redex, the only possibility is $r = S\langle v\rangle u$ for some abstraction v and substitution context S. By point 1, $E_1\langle x\rangle \neq S\langle v\rangle$, so this case is impossible.
  (b) $E' = E'_1 u$. Then it follows from the *i.h.* (on E_1).
- $E = E_1\langle y{\leftarrow}v\rangle$: two sub-cases.
  - $E' = \langle\cdot\rangle$: this time, the fact that r is a redex forces $r = E'_1\langle\!\langle y\rangle\!\rangle\langle y{\leftarrow}v\rangle$. Note that $E_1\langle\!\langle x\rangle\!\rangle = E'_1\langle\!\langle y\rangle\!\rangle$ so that by Point 2 the two decompositions coincide. This is absurd because then $x \neq y$ while by hypothesis E does not capture x.
  - $E' = E'_1\langle y{\leftarrow}u\rangle$. Then it follows from the *i.h.*
- $E = E_1[y{\leftarrow}u]$: three sub-cases.
  - $E' = \langle\cdot\rangle$: this time, the fact that r is a redex forces $r = E'_1\langle\!\langle y\rangle\!\rangle[y{\leftarrow}u]$ and $u = S\langle v\rangle$. Note that $E_1\langle\!\langle x\rangle\!\rangle = E'_1\langle\!\langle y\rangle\!\rangle$ so that by Point 2 the two decompositions coincide. This is absurd because then $x \neq y$ while by hypothesis E does not capture x.
  - $E' = E'_1[y{\leftarrow}u]$: it follows from the *i.h.* on E_1.
  - $E' = E'_1\langle\!\langle y\rangle\!\rangle[y{\leftarrow}E'_2]$ with $E'_2\langle r\rangle = u$: note that $E_1\langle\!\langle x\rangle\!\rangle = E'_1\langle\!\langle y\rangle\!\rangle$ so that by Point 2 the two decompositions coincide. This is absurd because then $x \neq y$ while by hypothesis E does not capture x.
- $E = E_1\langle y\rangle[y{\leftarrow}E_2]$: three sub-cases.
  - $E' = \langle\cdot\rangle$: the fact that r is a redex implies $r = E'_1\langle y\rangle[y{\leftarrow}S\langle v\rangle]$. This case is impossible: by Point 2, $E'_1 = E_1$, which implies $E_2\langle x\rangle \neq S\langle v\rangle$, that contradicts Point 1.
  - $E' = E'_1[y{\leftarrow}E_2\langle x\rangle]$: the specular case has already been treated above.
  - $E' = E_1\langle y\rangle[y{\leftarrow}E'_2]$: it follows from the *i.h.* on E_2. □

Proposition 3.3. *The reduction $\to_{\mathtt{sk\cdot need}}$ is deterministic.*

Originally at p. 7

Proof. Let t be a term that decomposes in two ways as an evaluation context surrounding a root redex, that is, $t = E_1\langle r_1\rangle = E_2\langle r_2\rangle$. We prove that $E_1 = E_2$ and $r_1 = r_2$ by structural induction on t. Cases:

- *Variable or abstraction* Impossible, since variables and abstractions are both call-by-need normal.
- *Application,* i.e. $t = us$ Suppose that one of the two evaluation contexts, for instance E_1, is equal to $\langle\cdot\rangle$. Then, we must have $u = \lambda x.u'$, but in that case it is easy to see that the result holds, because E_2 cannot have its hole to the right of an application (in s) or under an abstraction (in u'). We may then assume that none of E_1, E_2 is equal to $\langle\cdot\rangle$. Then, they have to have shape $E_1 = E'_1 s$ and $E_2 = E'_2 s$. The required equalities then are given by the *i.h.*
- *Skeletal substitution,* i.e. $t = u\langle x{\leftarrow}v\rangle$ Cases:

– *Both contexts have their holes in u.* It follows from the *i.h.*.

– *One of the contexts—say E_1—is empty,* i.e. $u = E_3\langle\!\langle x \rangle\!\rangle$, $s = v$, *and* $r_1 = E_3\langle\!\langle x \rangle\!\rangle\langle x{\leftarrow}v\rangle$. Then E_2 is also empty because $E\langle\!\langle x \rangle\!\rangle$ is normal by Lemma A.1.3.

- *Explicit substitution,* i.e. $t = u[x{\leftarrow}s]$ Cases:

  – *Both contexts have their holes in u or s.* It follows from the *i.h.*.

  – *One of the contexts—say E_1—is empty,* i.e. $u = E_3\langle\!\langle x \rangle\!\rangle$, $s = S\langle v\rangle$, *and* $r_1 = E_3\langle\!\langle x \rangle\!\rangle[x{\leftarrow}S\langle v\rangle]$. Then E_2 is also empty. Indeed:

    1. The hole of E_2 cannot be in $S\langle v\rangle$, because it is normal, and
    2. It cannot be inside $E_3\langle\!\langle x \rangle\!\rangle$ because by Lemma A.1.3 $E\langle\!\langle x \rangle\!\rangle$ is normal.

  – *One of the contexts—say E_1—has its hole in s and the other one has its hole in u,* i.e. $E_1 = E_3\langle\!\langle x \rangle\!\rangle[x{\leftarrow}E_4]$ *and* $E_2 = E_5[x{\leftarrow}s]$. This case is impossible, because by Lemma A.1.3 $E_3\langle\!\langle x \rangle\!\rangle$ is normal. □

B Proofs from Sect. 4

Lemma 4.1. *Let $n \in \mathbb{N}$ and $s_n := (x\mathtt{I}(x\mathtt{I}))[x{\leftarrow}u_n]$. Then there is a substitution context $S_n^{\mathtt{sk}}$ and an evaluation $\mathsf{e}_n : s_n \to^*_{\mathtt{sk{\cdot}need}} S_n^{\mathtt{sk}}\langle\mathtt{I}\rangle$ such that $|\mathsf{e}_n|_{\mathtt{sk{\cdot}dB}} = 6n{+}3$ and $|S_n^{\mathtt{sk}}| = \mathcal{O}(n)$.*

Originally at p. 9

Proof. By induction on n.

- **Case $n = 0$.**

$$\begin{array}{lll} s_0 & = & (x\mathtt{I}(x\mathtt{I}))[x{\leftarrow}\mathtt{I}] \\ & \to_{\mathtt{sk}} & (x\mathtt{I}(x\mathtt{I}))\langle x{\leftarrow}\mathtt{I}\rangle \\ & \to_{\mathtt{ss}} & (\mathtt{I}\mathtt{I}(x\mathtt{I}))\langle x{\leftarrow}\mathtt{I}\rangle \\ & \to_{\mathtt{sk{\cdot}dB}} & a[a{\leftarrow}\mathtt{I}](x\mathtt{I})\langle x{\leftarrow}\mathtt{I}\rangle \\ & \to_{\mathtt{sk}} & a\langle a{\leftarrow}\mathtt{I}\rangle(x\mathtt{I})\langle x{\leftarrow}\mathtt{I}\rangle \\ & \to_{\mathtt{ss}} & \mathtt{I}\langle a{\leftarrow}\mathtt{I}\rangle(x\mathtt{I})\langle x{\leftarrow}\mathtt{I}\rangle \\ & \to_{\mathtt{sk{\cdot}dB}} & b[b{\leftarrow}x\mathtt{I}]\langle a{\leftarrow}\mathtt{I}\rangle\langle x{\leftarrow}\mathtt{I}\rangle \\ & \to_{\mathtt{ss}} & b[b{\leftarrow}\mathtt{I}\mathtt{I}]\langle a{\leftarrow}\mathtt{I}\rangle\langle x{\leftarrow}\mathtt{I}\rangle \\ & \to_{\mathtt{sk{\cdot}dB}} & b[b{\leftarrow}c[c{\leftarrow}\mathtt{I}]]\langle a{\leftarrow}\mathtt{I}\rangle\langle x{\leftarrow}\mathtt{I}\rangle \\ & \to_{\mathtt{sk}}\to_{\mathtt{ss}} & b[b{\leftarrow}\mathtt{I}\langle c{\leftarrow}\mathtt{I}\rangle]\langle a{\leftarrow}\mathtt{I}\rangle\langle x{\leftarrow}\mathtt{I}\rangle \\ & \to_{\mathtt{sk}}\to_{\mathtt{ss}} & \mathtt{I}\langle b{\leftarrow}\mathtt{I}\rangle\langle c{\leftarrow}\mathtt{I}\rangle\langle a{\leftarrow}\mathtt{I}\rangle\langle x{\leftarrow}\mathtt{I}\rangle \end{array}$$

  Then, setting $S_0^{\mathtt{sk}} := \langle\cdot\rangle\langle b{\leftarrow}\mathtt{I}\rangle\langle c{\leftarrow}\mathtt{I}\rangle\langle a{\leftarrow}\mathtt{I}\rangle\langle x{\leftarrow}\mathtt{I}\rangle$, we obtain that $s_0 \to^*_{\mathtt{sk{\cdot}need}} S_0^{\mathtt{sk}}\langle\mathtt{I}\rangle$ using 3 $\to_{\mathtt{sk{\cdot}dB}}$-steps.

- **Case $n + 1$.**

$$\begin{array}{lll} s_{n+1} & = & (x\mathtt{I}(x\mathtt{I}))[x{\leftarrow}u_{n+1}] \\ & = & (x\mathtt{I}(x\mathtt{I}))[x{\leftarrow}(\lambda y.\lambda z.y\mathtt{I}(y\mathtt{I})z)u_n] \\ & \to_{\mathtt{sk{\cdot}dB}} & (x\mathtt{I}(x\mathtt{I}))[x{\leftarrow}(\lambda z.y\mathtt{I}(y\mathtt{I})z)[y{\leftarrow}u_n]] \\ & \to_{\mathtt{sk}} & (x\mathtt{I}(x\mathtt{I}))\langle x{\leftarrow}\lambda z.wz\rangle[w{\leftarrow}y\mathtt{I}(y\mathtt{I})][y{\leftarrow}u_n] \\ & \to_{\mathtt{ss}} & ((\lambda z.wz)\mathtt{I})(x\mathtt{I})\langle x{\leftarrow}\lambda z.wz\rangle[w{\leftarrow}y\mathtt{I}(y\mathtt{I})][y{\leftarrow}u_n] \\ & \to_{\mathtt{sk{\cdot}dB}} & (wz)[z{\leftarrow}\mathtt{I}](x\mathtt{I})\langle x{\leftarrow}\lambda z.wz\rangle[w{\leftarrow}y\mathtt{I}(y\mathtt{I})][y{\leftarrow}u_n] \\ & \equiv & (wz)[z{\leftarrow}\mathtt{I}](x\mathtt{I})\langle x{\leftarrow}\lambda z.wz\rangle[w{\leftarrow}y\mathtt{I}(y\mathtt{I})[y{\leftarrow}u_n]] \end{array}$$

Please note the use of structural equivalence at the last step. Now, consider the context $E := (wz)[z\leftarrow \mathtt{I}](x\mathtt{I})\langle x\leftarrow \lambda z.wz\rangle[w\leftarrow\langle\cdot\rangle]$. By *i.h.*, there is $S_n^{\mathtt{sk}}$ such that:

$$E\langle\!\langle y\mathtt{I}(y\mathtt{I})[y\leftarrow u_n]\rangle\!\rangle \quad \to^*_{\mathtt{sk\cdot need}} \quad E\langle\!\langle S_n^{\mathtt{sk}}\langle\mathtt{I}\rangle\rangle\!\rangle$$

by doing $6n+3$ $\mathtt{dB}$-step. The evaluation continue as follows:

$$\begin{array}{lll}
E\langle\!\langle S_n^{\mathtt{sk}}\langle\mathtt{I}\rangle\rangle\!\rangle & = & (wz)[z\leftarrow\mathtt{I}](x\mathtt{I})\langle x\leftarrow\lambda z.wz\rangle[w\leftarrow S_n^{\mathtt{sk}}\langle\mathtt{I}\rangle] \\
& \to_{\mathtt{sk}} & S_n^{\mathtt{sk}}\langle(wz)[z\leftarrow\mathtt{I}](x\mathtt{I})\langle x\leftarrow\lambda z.wz\rangle\langle w\leftarrow\mathtt{I}\rangle\rangle \\
& \to_{\mathtt{ss}} & S_n^{\mathtt{sk}}\langle(\mathtt{I}z)[z\leftarrow\mathtt{I}](x\mathtt{I})\langle x\leftarrow\lambda z.wz\rangle\langle w\leftarrow\mathtt{I}\rangle\rangle \\
& \to_{\mathtt{sk\cdot dB}} & S_n^{\mathtt{sk}}\langle a[a\leftarrow z][z\leftarrow\mathtt{I}](x\mathtt{I})\langle x\leftarrow\lambda z.wz\rangle\langle w\leftarrow\mathtt{I}\rangle\rangle \\
& \to_{\mathtt{sk}}\to_{\mathtt{ss}} & S_n^{\mathtt{sk}}\langle a[a\leftarrow\mathtt{I}]\langle z\leftarrow\mathtt{I}\rangle(x\mathtt{I})\langle x\leftarrow\lambda z.wz\rangle\langle w\leftarrow\mathtt{I}\rangle\rangle \\
& \to_{\mathtt{sk}}\to_{\mathtt{ss}} & S_n^{\mathtt{sk}}\langle\mathtt{I}\langle a\leftarrow\mathtt{I}\rangle\langle z\leftarrow\mathtt{I}\rangle(x\mathtt{I})\langle x\leftarrow\lambda z.wz\rangle\langle w\leftarrow\mathtt{I}\rangle\rangle \\
& \to_{\mathtt{sk\cdot dB}} & S_n^{\mathtt{sk}}\langle b[b\leftarrow x\mathtt{I}]\langle a\leftarrow\mathtt{I}\rangle\langle z\leftarrow\mathtt{I}\rangle\langle x\leftarrow\lambda z.wz\rangle\langle w\leftarrow\mathtt{I}\rangle\rangle \\
& \to_{\mathtt{ss}} & S_n^{\mathtt{sk}}\langle b[b\leftarrow(\lambda z'.wz')\mathtt{I}]\langle a\leftarrow\mathtt{I}\rangle\langle z\leftarrow\mathtt{I}\rangle\langle x\leftarrow\lambda z.wz\rangle\langle w\leftarrow\mathtt{I}\rangle\rangle \\
& \to_{\mathtt{sk\cdot dB}} & S_n^{\mathtt{sk}}\langle b[b\leftarrow(wz')[z'\leftarrow\mathtt{I}]]\langle a\leftarrow\mathtt{I}\rangle\langle z\leftarrow\mathtt{I}\rangle\langle x\leftarrow\lambda z.wz\rangle\langle w\leftarrow\mathtt{I}\rangle\rangle \\
& \to_{\mathtt{ss}} & S_n^{\mathtt{sk}}\langle b[b\leftarrow(\mathtt{I}z')[z'\leftarrow\mathtt{I}]]\langle a\leftarrow\mathtt{I}\rangle\langle z\leftarrow\mathtt{I}\rangle\langle x\leftarrow\lambda z.wz\rangle\langle w\leftarrow\mathtt{I}\rangle\rangle \\
& \to_{\mathtt{sk\cdot dB}} & S_n^{\mathtt{sk}}\langle b[b\leftarrow c[c\leftarrow z'][z'\leftarrow\mathtt{I}]]\langle a\leftarrow\mathtt{I}\rangle\langle z\leftarrow\mathtt{I}\rangle\langle x\leftarrow\lambda z.wz\rangle\langle w\leftarrow\mathtt{I}\rangle\rangle \\
& \to_{\mathtt{sk}}\to_{\mathtt{ss}} & S_n^{\mathtt{sk}}\langle b[b\leftarrow c[c\leftarrow\mathtt{I}]\langle z'\leftarrow\mathtt{I}\rangle]\langle a\leftarrow\mathtt{I}\rangle\langle z\leftarrow\mathtt{I}\rangle\langle x\leftarrow\lambda z.wz\rangle\langle w\leftarrow\mathtt{I}\rangle\rangle \\
& \to_{\mathtt{sk}}\to_{\mathtt{ss}} & S_n^{\mathtt{sk}}\langle b[b\leftarrow\mathtt{I}\langle c\leftarrow\mathtt{I}\rangle\langle z'\leftarrow\mathtt{I}\rangle]\langle a\leftarrow\mathtt{I}\rangle\langle z\leftarrow\mathtt{I}\rangle\langle x\leftarrow\lambda z.wz\rangle\langle w\leftarrow\mathtt{I}\rangle\rangle \\
& \to_{\mathtt{sk}}\to_{\mathtt{ss}} & S_n^{\mathtt{sk}}\langle\mathtt{I}\langle b\leftarrow\mathtt{I}\rangle\langle c\leftarrow\mathtt{I}\rangle\langle z'\leftarrow\mathtt{I}\rangle\langle a\leftarrow\mathtt{I}\rangle\langle z\leftarrow\mathtt{I}\rangle\langle x\leftarrow\lambda z.wz\rangle\langle w\leftarrow\mathtt{I}\rangle\rangle
\end{array}$$

Then, let $S_\bullet := S_n^{\mathtt{sk}}\langle\langle\cdot\rangle\langle b\leftarrow\mathtt{I}\rangle\langle c\leftarrow\mathtt{I}\rangle\langle z'\leftarrow\mathtt{I}\rangle\langle a\leftarrow\mathtt{I}\rangle\langle z\leftarrow\mathtt{I}\rangle\langle x\leftarrow\lambda z.wz\rangle\langle w\leftarrow\mathtt{I}\rangle\rangle$, so that we can write $E\langle\!\langle S_n^{\mathtt{sk}}\langle\mathtt{I}\rangle\rangle\!\rangle \to^*_{\mathtt{sk\cdot need}} S_\bullet\langle\mathtt{I}\rangle$. Now, by putting all together, we obtain:

$$s_{n+1} \quad \to^*_{\mathtt{sk\cdot need}} \equiv E\langle\!\langle y\mathtt{I}(y\mathtt{I})[y\leftarrow u_n]\rangle\!\rangle \quad \to^*_{\mathtt{sk\cdot need}} \quad E\langle\!\langle S_n^{\mathtt{sk}}\langle\mathtt{I}\rangle\rangle\!\rangle \quad \to^*_{\mathtt{sk\cdot need}} \quad S_\bullet\langle\mathtt{I}\rangle$$

using $2+(6n+3)+4 = 6(n+1)+3$ $\mathtt{sk\cdot dB}$-steps. By strong bisimulation of $\equiv$ (Prop. 3.4), we obtain, for some r, the following re-organization of the previous sequence, having in particular the same number and kind of steps:

$$s_{n+1} \quad \to^*_{\mathtt{sk\cdot need}} \quad r \quad \equiv \quad S_\bullet\langle\mathtt{I}\rangle$$

Note also that r has shape $S_\circ\langle\mathtt{I}\rangle$ for some substitution context $S_\circ$, because $\equiv$ cannot move anything in/out of abstractions, and that $|S_\circ| = |S_\bullet|$ because $\equiv$ preserves the number of constructors. Therefor, by setting $S_{n+1}^{\mathtt{sk}} := S_\circ$, we obtain a sequence $s_{n+1} \to^*_{\mathtt{sk\cdot need}} S_{n+1}^{\mathtt{sk}}\langle\mathtt{I}\rangle$ satisfying the statement (and free from $\equiv$).

The fact that $|S_n^{\mathtt{sk}}| = \mathcal{O}(n)$ is evident from the definition of $S_\bullet$, which uses the n case linearly. □

Evaluation in CbNeed. The family of contexts $S_n^\bullet$ in the statement of the next lemma is defined in its proof.

Lemma B.1. *Let $n \in \mathbb{N}$ and E be an evaluation context. Then there exist a substitution context $S_n^\bullet$ and an evaluation $\mathtt{e}_n : S_n\langle E\langle\!\langle x_n\mathtt{I}(x_n\mathtt{I})\rangle\!\rangle\rangle \to^*_{\mathtt{need}} S_n\langle E\langle\!\langle S_n^\bullet\langle\mathtt{I}\rangle\rangle\!\rangle\rangle$ such that $|\mathtt{e}_n|_{\mathtt{dB}} = 8\times 2^n - 5$ and $|S_n^\bullet| = \Omega(2^n)$.*

Originally at p. 23

Proof. First of all, we recall the definition of S_n from the body of the paper:

- $S_0 := \langle\cdot\rangle[x_0{\leftarrow}\mathtt{I}]$, and
- $S_{n+1} := S_n\langle\langle\cdot\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle$ with $s_{n+1} := \lambda y_n.x_n\mathtt{I}(x_n\mathtt{I})y_n$.

By induction on n.

- **Case** $n = 0$**.**

$$\begin{array}{lll} S_0\langle E\langle\!\langle x_0\mathtt{I}(x_0\mathtt{I})\rangle\!\rangle\rangle & = & E\langle\!\langle x_0\mathtt{I}(x_0\mathtt{I})\rangle\!\rangle[x_0{\leftarrow}\mathtt{I}] \\ & \to_{\mathtt{nd{\cdot}ls}} & E\langle\!\langle \mathtt{I}\mathtt{I}(x_0\mathtt{I})\rangle\!\rangle[x_0{\leftarrow}\mathtt{I}] \\ & \to_{\mathtt{dB}} & E\langle\!\langle a[a{\leftarrow}\mathtt{I}](x_0\mathtt{I})\rangle\!\rangle[x_0{\leftarrow}\mathtt{I}] \\ & \to_{\mathtt{nd{\cdot}ls}} & E\langle\!\langle \mathtt{I}[a{\leftarrow}\mathtt{I}](x_0\mathtt{I})\rangle\!\rangle[x_0{\leftarrow}\mathtt{I}] \\ & \to_{\mathtt{dB}} & E\langle\!\langle b[b{\leftarrow}x_0\mathtt{I}][a{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_0{\leftarrow}\mathtt{I}] \\ & \to_{\mathtt{nd{\cdot}ls}} & E\langle\!\langle b[b{\leftarrow}\mathtt{I}\mathtt{I}][a{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_0{\leftarrow}\mathtt{I}] \\ & \to_{\mathtt{dB}} & E\langle\!\langle b[b{\leftarrow}c[c{\leftarrow}\mathtt{I}]][a{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_0{\leftarrow}\mathtt{I}] \\ & \to_{\mathtt{nd{\cdot}ls}}\to_{\mathtt{nd{\cdot}ls}} & E\langle\!\langle \mathtt{I}[b{\leftarrow}\mathtt{I}][c{\leftarrow}\mathtt{I}][a{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_0{\leftarrow}\mathtt{I}] \end{array}$$

  Then, setting $S_0^\bullet := \langle\cdot\rangle[b{\leftarrow}\mathtt{I}][c{\leftarrow}\mathtt{I}][a{\leftarrow}\mathtt{I}]$, we obtain that $S_0\langle E\langle\!\langle x_0\mathtt{I}(x_0\mathtt{I})\rangle\!\rangle\rangle \to^*_{\mathtt{need}} S_0\langle E\langle\!\langle S_0^\bullet\langle\mathtt{I}\rangle\rangle\!\rangle\rangle$ using 3 $\to_{\mathtt{dB}}$-steps.

- **Case** $n + 1$**.**

$$\begin{array}{lll} S_{n+1}\langle E\langle\!\langle x_{n+1}\mathtt{I}(x_{n+1}\mathtt{I})\rangle\!\rangle\rangle & = & S_n\langle E\langle\!\langle x_{n+1}\mathtt{I}(x_{n+1}\mathtt{I})\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle \\ & \to_{\mathtt{nd{\cdot}ls}} & S_n\langle E\langle\!\langle (\lambda y_n.x_n\mathtt{I}(x_n\mathtt{I})y_n)\mathtt{I}(x_{n+1}\mathtt{I})\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle \\ & \to_{\mathtt{dB}} & S_n\langle E\langle\!\langle (x_n\mathtt{I}(x_n\mathtt{I})y_n)[y_n{\leftarrow}\mathtt{I}](x_{n+1}\mathtt{I})\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle \end{array}$$

  We have a context $E' := E\langle\!\langle(\langle\cdot\rangle y_n)[y_n{\leftarrow}\mathtt{I}](x_{n+1}\mathtt{I})\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]$. By *i.h.*, there is $S_n^\bullet$ such that $S_n\langle E'\langle\!\langle x_n\mathtt{I}(x_n\mathtt{I})\rangle\!\rangle\rangle \to^*_{\mathtt{need}} S_n\langle E'\langle\!\langle S_n^\bullet\langle\mathtt{I}\rangle\rangle\!\rangle\rangle$ by doing $8 \times 2^n - 5$ $\mathtt{dB}$-steps. The evaluation continues as follows:

$$\begin{array}{lll} S_n\langle E'\langle\!\langle S_n^\bullet\langle\mathtt{I}\rangle\rangle\!\rangle\rangle & = & S_n\langle E\langle\!\langle (S_n^\bullet\langle\mathtt{I}\rangle y_n)[y_n{\leftarrow}\mathtt{I}](x_{n+1}\mathtt{I})\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle \\ & \to_{\mathtt{dB}} & S_n\langle E\langle\!\langle S_n^\bullet\langle a[a{\leftarrow}y_n]\rangle[y_n{\leftarrow}\mathtt{I}](x_{n+1}\mathtt{I})\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle \\ & \to_{\mathtt{nd{\cdot}ls}}\to_{\mathtt{nd{\cdot}ls}} & S_n\langle E\langle\!\langle S_n^\bullet\langle\mathtt{I}[a{\leftarrow}\mathtt{I}][y_n{\leftarrow}\mathtt{I}]\rangle(x_{n+1}\mathtt{I})\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle \\ & \to_{\mathtt{dB}} & S_n\langle E\langle\!\langle S_n^\bullet\langle b[b{\leftarrow}x_{n+1}\mathtt{I}][a{\leftarrow}\mathtt{I}][y_n{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle \\ & \to_{\mathtt{nd{\cdot}ls}} & S_n\langle E\langle\!\langle S_n^\bullet\langle b[b{\leftarrow}(\lambda y_n.x_n\mathtt{I}(x_n\mathtt{I})y_n)\mathtt{I}][a{\leftarrow}\mathtt{I}][y_n{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle \\ & \to_{\mathtt{dB}} & S_n\langle E\langle\!\langle S_n^\bullet\langle b[b{\leftarrow}(x_n\mathtt{I}(x_n\mathtt{I})y_n)[y_n{\leftarrow}\mathtt{I}]][a{\leftarrow}\mathtt{I}][y_n{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle \end{array}$$

  We have a context $E'' = E\langle\!\langle S_n^\bullet\langle b[b{\leftarrow}(\langle\cdot\rangle y_n)[y_n{\leftarrow}\mathtt{I}]][a{\leftarrow}\mathtt{I}][y_n{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]$. By *i.h.*, there is $S_n^\bullet$ such that $S_n\langle E''\langle\!\langle x_n\mathtt{I}(x_n\mathtt{I})\rangle\!\rangle\rangle \to^*_{\mathtt{need}} S_n\langle E''\langle\!\langle S_n^\bullet\langle\mathtt{I}\rangle\rangle\!\rangle\rangle$ by doing $8 \times 2^n - 5$ $\mathtt{dB}$-steps. The evaluation continues as follows:

$$\begin{array}{lll} S_n\langle E''\langle\!\langle S_n^\bullet\langle\mathtt{I}\rangle\rangle\!\rangle\rangle & = & S_n\langle E\langle\!\langle S_n^\bullet\langle b[b{\leftarrow}(S_n^\bullet\langle\mathtt{I}\rangle y_n)[y_n{\leftarrow}\mathtt{I}]][a{\leftarrow}\mathtt{I}][y_n{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle \\ & \to_{\mathtt{dB}} & S_n\langle E\langle\!\langle S_n^\bullet\langle b[b{\leftarrow}S_n^\bullet\langle c[c{\leftarrow}y_n]\rangle[y_n{\leftarrow}\mathtt{I}]][a{\leftarrow}\mathtt{I}][y_n{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle \\ & \to_{\mathtt{nd{\cdot}ls}}\to_{\mathtt{nd{\cdot}ls}} & S_n\langle E\langle\!\langle S_n^\bullet\langle b[b{\leftarrow}S_n^\bullet\langle\mathtt{I}[c{\leftarrow}\mathtt{I}]\rangle[y_n{\leftarrow}\mathtt{I}]][a{\leftarrow}\mathtt{I}][y_n{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle \\ & \to_{\mathtt{nd{\cdot}ls}} & S_n\langle E\langle\!\langle S_n^\bullet\langle S_n^\bullet\langle\mathtt{I}[b{\leftarrow}\mathtt{I}][c{\leftarrow}\mathtt{I}]\rangle[y_n{\leftarrow}\mathtt{I}][a{\leftarrow}\mathtt{I}][y_n{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle \end{array}$$

  Then, setting $S_{n+1}^\bullet := S_n^\bullet\langle S_n^\bullet\langle\langle\cdot\rangle[b{\leftarrow}\mathtt{I}][c{\leftarrow}\mathtt{I}]\rangle[y_n{\leftarrow}\mathtt{I}][a{\leftarrow}\mathtt{I}][y_n{\leftarrow}\mathtt{I}]\rangle$, we obtain that $S_0\langle E\langle\!\langle x_0\mathtt{I}(x_0\mathtt{I})\rangle\!\rangle\rangle \to^*_{\mathtt{need}} S_{n+1}\langle E\langle\!\langle {S''}_n^\bullet\langle\mathtt{I}\rangle\rangle\!\rangle\rangle$ using $1 + (8 \times 2^n - 5) + 3 + (8 \times 2^n - 5) + 1 = 8 \times 2^{n+1} - 5$ $\mathtt{dB}$-steps.

The fact that $|S_n^\bullet| = \Omega(2^n)$ is evident from the definition of $S_n^\bullet$, which for $n + 1$ uses twice the n case. □

Lemma 4.2. *Let $n \in \mathbb{N}$ and E be an evaluation context. Then there exist a substitution context $S_n^{\mathtt{nd}}$ and an evaluation $\mathtt{e}_n : E\langle\!\langle x_n\mathtt{I}(x_n\mathtt{I})\rangle\!\rangle[x_n{\leftarrow}u_n] \to^*_{\mathtt{need}} S_n\langle E\langle\!\langle S_n^{\mathtt{nd}}\langle\mathtt{I}\rangle\rangle\!\rangle\rangle$ such that $|\mathtt{e}_n|_{\mathtt{dB}} = 8\times 2^n + n - 5$ and $|S_n^{\mathtt{nd}}| = \Omega(2^n)$.*

Originally at p. 9

Proof. First of all, we recall the definition of S_n from the body of the paper:

- $S_0 := \langle\cdot\rangle[x_0{\leftarrow}\mathtt{I}]$, and
- $S_{n+1} := S_n\langle\langle\cdot\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle$ with $s_{n+1} := \lambda y_n.x_n\mathtt{I}(x_n\mathtt{I})y_n$.

By induction on n.

- **Case $n = 0$.**

$$\begin{array}{lll}
E\langle\!\langle x_0\mathtt{I}(x_0\mathtt{I})\rangle\!\rangle[x_0{\leftarrow}u_0] & = & E\langle\!\langle x_0\mathtt{I}(x_0\mathtt{I})\rangle\!\rangle[x_0{\leftarrow}\mathtt{I}]\\
& \to_{\mathtt{nd{\cdot}ls}} & E\langle\!\langle \mathtt{I}\mathtt{I}(x_0\mathtt{I})\rangle\!\rangle[x_0{\leftarrow}\mathtt{I}]\\
& \to_{\mathtt{dB}} & E\langle\!\langle a[a{\leftarrow}\mathtt{I}](x_0\mathtt{I})\rangle\!\rangle[x_0{\leftarrow}\mathtt{I}]\\
& \to_{\mathtt{nd{\cdot}ls}} & E\langle\!\langle \mathtt{I}[a{\leftarrow}\mathtt{I}](x_0\mathtt{I})\rangle\!\rangle[x_0{\leftarrow}\mathtt{I}]\\
& \to_{\mathtt{dB}} & E\langle\!\langle b[b{\leftarrow}x_0\mathtt{I}][a{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_0{\leftarrow}\mathtt{I}]\\
& \to_{\mathtt{nd{\cdot}ls}} & E\langle\!\langle b[b{\leftarrow}\mathtt{I}\mathtt{I}][a{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_0{\leftarrow}\mathtt{I}]\\
& \to_{\mathtt{dB}} & E\langle\!\langle b[b{\leftarrow}c[c{\leftarrow}\mathtt{I}]][a{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_0{\leftarrow}\mathtt{I}]\\
& \to_{\mathtt{nd{\cdot}ls}}\to_{\mathtt{nd{\cdot}ls}} & E\langle\!\langle \mathtt{I}[b{\leftarrow}\mathtt{I}][c{\leftarrow}\mathtt{I}][a{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_0{\leftarrow}\mathtt{I}]
\end{array}$$

  Then, setting $S_0^{\mathtt{nd}} := \langle\cdot\rangle[b{\leftarrow}\mathtt{I}][c{\leftarrow}\mathtt{I}][a{\leftarrow}\mathtt{I}]$, we obtain that $E\langle\!\langle x_0\mathtt{I}(x_0\mathtt{I})\rangle\!\rangle[x_0{\leftarrow}u_0] \to^*_{\mathtt{need}} S_0\langle E\langle\!\langle S_0^{\mathtt{nd}}\langle\mathtt{I}\rangle\rangle\!\rangle\rangle$ using 3 $\mathtt{dB}$-steps.

- **Case $n + 1$.**

$$\begin{array}{lll}
E\langle\!\langle x_{n+1}\mathtt{I}(x_{n+1}\mathtt{I})\rangle\!\rangle[x_{n+1}{\leftarrow}u_{n+1}] & = & E\langle\!\langle x_{n+1}\mathtt{I}(x_{n+1}\mathtt{I})\rangle\!\rangle[x_{n+1}{\leftarrow}(\lambda x_n.\lambda y_n.(x_n\mathtt{I})(x_n\mathtt{I})y_n)u_n]\\
& \to_{\mathtt{dB}} & E\langle\!\langle x_{n+1}\mathtt{I}(x_{n+1}\mathtt{I})\rangle\!\rangle[x_{n+1}{\leftarrow}(\lambda y_n.x_n\mathtt{I}(x_n\mathtt{I})y_n)[x_n{\leftarrow}u_n]]\\
& \to_{\mathtt{nd{\cdot}ls}} & E\langle\!\langle (\lambda y_n.x_n\mathtt{I}(x_n\mathtt{I})y_n)\mathtt{I}(x_{n+1}\mathtt{I})\rangle\!\rangle[x_{n+1}{\leftarrow}\lambda y_n.x_n\mathtt{I}(x_n\mathtt{I})y_n][x_n{\leftarrow}\\
& \to_{\mathtt{dB}} & E\langle\!\langle (x_n\mathtt{I}(x_n\mathtt{I})y_n)[y_n{\leftarrow}\mathtt{I}](x_{n+1}\mathtt{I})\rangle\!\rangle[x_{n+1}{\leftarrow}\lambda y_n.x_n\mathtt{I}(x_n\mathtt{I})y_n][x_n
\end{array}$$

  Consider the context $E' := E\langle\!\langle(\langle\cdot\rangle y_n)[y_n{\leftarrow}\mathtt{I}](x_{n+1}\mathtt{I})\rangle\!\rangle[x_{n+1}{\leftarrow}\lambda y_n.x_n\mathtt{I}(x_n\mathtt{I})y_n]$.
  By *i.h.*, there is $S_n^{\mathtt{nd}}$ such that $E'\langle\!\langle x_n\mathtt{I}(x_n\mathtt{I})\rangle\!\rangle[x_n{\leftarrow}u_n] \to^*_{\mathtt{need}} S_n\langle E'\langle\!\langle S_n^{\mathtt{nd}}\langle\mathtt{I}\rangle\rangle\!\rangle\rangle$
  by doing $8\times 2^n - 5$ $\mathtt{dB}$-steps. The evaluation continues as follows:

$$\begin{array}{lll}
S_n\langle E'\langle\!\langle S_n^{\mathtt{nd}}\langle\mathtt{I}\rangle\rangle\!\rangle\rangle & = & S_n\langle E\langle\!\langle (S_n^{\mathtt{nd}}\langle\mathtt{I}\rangle y_n)[y_n{\leftarrow}\mathtt{I}](x_{n+1}\mathtt{I})\rangle\!\rangle[x_{n+1}{\leftarrow}\lambda y_n.x_n\mathtt{I}(x_n\mathtt{I})y_n]\rangle\\
& = & S_n\langle E\langle\!\langle (S_n^{\mathtt{nd}}\langle\mathtt{I}\rangle y_n)[y_n{\leftarrow}\mathtt{I}](x_{n+1}\mathtt{I})\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle\\
& \to_{\mathtt{dB}} & S_n\langle E\langle\!\langle S_n^{\mathtt{nd}}\langle a[a{\leftarrow}y_n]\rangle[y_n{\leftarrow}\mathtt{I}](x_{n+1}\mathtt{I})\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle\\
& \to_{\mathtt{nd{\cdot}ls}}\to_{\mathtt{nd{\cdot}ls}} & S_n\langle E\langle\!\langle S_n^{\mathtt{nd}}\langle \mathtt{I}[a{\leftarrow}\mathtt{I}]\rangle[y_n{\leftarrow}\mathtt{I}](x_{n+1}\mathtt{I})\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle\\
& \to_{\mathtt{dB}} & S_n\langle E\langle\!\langle S_n^{\mathtt{nd}}\langle b[b{\leftarrow}x_{n+1}\mathtt{I}][a{\leftarrow}\mathtt{I}]\rangle[y_n{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle\\
& \to_{\mathtt{nd{\cdot}ls}} & S_n\langle E\langle\!\langle S_n^{\mathtt{nd}}\langle b[b{\leftarrow}(\lambda y'_n.x_n\mathtt{I}(x_n\mathtt{I})y'_n)\mathtt{I}][a{\leftarrow}\mathtt{I}]\rangle[y_n{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle\\
& \to_{\mathtt{dB}} & S_n\langle E\langle\!\langle S_n^{\mathtt{nd}}\langle b[b{\leftarrow}(x_n\mathtt{I}(x_n\mathtt{I})y'_n)[y'_n{\leftarrow}\mathtt{I}]][a{\leftarrow}\mathtt{I}]\rangle[y_n{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle
\end{array}$$

  Now, consider the context $E'' := E\langle\!\langle S_n^{\mathtt{nd}}\langle b[b{\leftarrow}(\langle\cdot\rangle y'_n)[y'_n{\leftarrow}\mathtt{I}]][a{\leftarrow}\mathtt{I}]\rangle[y_n{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]$.
  By Lemma B.1, there is $S_n^\bullet$ such that $S_n\langle E''\langle\!\langle x_n\mathtt{I}(x_n\mathtt{I})\rangle\!\rangle\rangle \to^*_{\mathtt{need}} S_n\langle E''\langle\!\langle S_n^\bullet\langle\mathtt{I}\rangle\rangle\!\rangle\rangle$
  by doing $8\times 2^n - 5$ $\mathtt{dB}$-steps. The evaluation continues as follows:

$$\begin{array}{lll}
S_n\langle E''\langle\!\langle S_n^\bullet\langle\mathtt{I}\rangle\rangle\!\rangle\rangle & = & S_n\langle E\langle\!\langle S_n^{\mathtt{nd}}\langle b[b{\leftarrow}(S_n^\bullet\langle\mathtt{I}\rangle y'_n)[y'_n{\leftarrow}\mathtt{I}]][a{\leftarrow}\mathtt{I}]\rangle[y_n{\leftarrow}\mathtt{I}]\rangle\!\rangle[x_{n+1}{\leftarrow}s_{n+1}]\rangle\\
& = & S_{n+1}\langle E\langle\!\langle S_n^{\mathtt{nd}}\langle b[b{\leftarrow}(S_n^\bullet\langle\mathtt{I}\rangle y'_n)[y'_n{\leftarrow}\mathtt{I}]][a{\leftarrow}\mathtt{I}]\rangle[y_n{\leftarrow}\mathtt{I}]\rangle\!\rangle\rangle\\
& \to_{\mathtt{dB}} & S_{n+1}\langle E\langle\!\langle S_n^{\mathtt{nd}}\langle b[b{\leftarrow}S_n^\bullet\langle c[c{\leftarrow}y'_n]\rangle[y'_n{\leftarrow}\mathtt{I}]][a{\leftarrow}\mathtt{I}]\rangle[y_n{\leftarrow}\mathtt{I}]\rangle\!\rangle\rangle\\
& \to_{\mathtt{nd{\cdot}ls}}\to_{\mathtt{nd{\cdot}ls}} & S_{n+1}\langle E\langle\!\langle S_n^{\mathtt{nd}}\langle b[b{\leftarrow}S_n^\bullet\langle \mathtt{I}[c{\leftarrow}\mathtt{I}]\rangle[y'_n{\leftarrow}\mathtt{I}]][a{\leftarrow}\mathtt{I}]\rangle[y_n{\leftarrow}\mathtt{I}]\rangle\!\rangle\rangle\\
& \to_{\mathtt{nd{\cdot}ls}} & S_{n+1}\langle E\langle\!\langle S_n^{\mathtt{nd}}\langle S_n^\bullet\langle \mathtt{I}[b{\leftarrow}\mathtt{I}][c{\leftarrow}\mathtt{I}]\rangle[y'_n{\leftarrow}\mathtt{I}][a{\leftarrow}\mathtt{I}]\rangle[y_n{\leftarrow}\mathtt{I}]\rangle\!\rangle\rangle
\end{array}$$

Then, setting $S^{\mathtt{nd}}_{n+1} := S^{\mathtt{nd}}_n\langle S^{\bullet}_n\langle\langle\cdot\rangle[b{\leftarrow}\mathtt{I}][c{\leftarrow}\mathtt{I}]\rangle[y'_n{\leftarrow}\mathtt{I}][a{\leftarrow}\mathtt{I}]\rangle[y_n{\leftarrow}\mathtt{I}]$, we obtain that:

$$E\langle\!\langle x_{n+1}\mathtt{I}(x_{n+1}\mathtt{I})\rangle\!\rangle[x_{n+1}{\leftarrow}u_{n+1}] \quad \to^{*}_{\mathtt{need}} \quad S_{n+1}\langle E\langle\!\langle S^{\mathtt{nd}}_n\langle\mathtt{I}\rangle\rangle\!\rangle\rangle$$

using $1+(8\times 2^n-5)+3+(8\times 2^n-5)+1 = 8\times 2^{n+1}-10+5+n+1 = 8\times 2^{n+1}+(n+1)-5$ $\mathtt{dB}$-steps.

The fact that $|S^{\mathtt{nd}}_n| = \Omega(2^n)$ follows from the fact that $|S^{\bullet}_n| = \Omega(2^n)$ by Lemma B.1. □

C Proofs about Marked Terms

Lemma 5.2 (Being unmarked is downward closed in marked skeletons)**.** *Let* $\mathtt{mskel}^{\mathtt{V}}(t) = M\langle m\rangle$ *with* m *starting unmarked. Then* m *has no marks.*

Originally at p. 11

Proof. Straightforward induction on M. □

Proposition 5.4 (Soundness of the marked skeleton)**.**
Let $t \in \Lambda$ *and* $\mathtt{V}$ *be a set of variables.*

1. Auxiliary parametrized statement*:* $\mathtt{split}(\mathtt{mskel}^{\mathtt{V}}(t)) = \mathtt{skdec}^{\mathtt{V}}(t)$*;*
2. Soundness*:* $\mathtt{skdec}(\lambda x.t) = \mathtt{split}(\mathtt{mskel}(\lambda x.t))$.

Originally at p. 11

Proof.

1. Straightforward induction on t. We give the details nonetheless. Cases:
   - **Case** $\mathtt{fv}(t)\cap\mathtt{V} = \emptyset$ **and** t **is not a variable.** Then by definition, $\mathtt{skdec}^{\mathtt{V}}(t) = (z, \langle\cdot\rangle[z{\leftarrow}t])$. While, $\mathtt{mskel}^{\mathtt{V}}(t) = t$ and $\mathtt{split}(\mathtt{mskel}^{\mathtt{V}}(t)) = (z, \langle\cdot\rangle[z{\leftarrow}t])$.
   - **Case** $t = x$**.** Then $\mathtt{mskel}^{\mathtt{V}}(t) = x$ or $\mathtt{mskel}^{\mathtt{V}}(t) = x^{\circ}$. In both cases, by definition, $\mathtt{split}(\mathtt{mskel}^{\mathtt{V}}(t)) = (x, \langle\cdot\rangle)$. By definition, $\mathtt{skdec}^{\mathtt{V}}(x) = (x, \langle\cdot\rangle)$.
   - **Case** $t = \lambda x.u$ **and** $\mathtt{fv}(t)\cap\mathtt{V} \neq \emptyset$**.** By definition $\mathtt{mskel}^{\mathtt{V}}(t) = \lambda^{\circ}x.\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(u)$ and $\mathtt{split}(\mathtt{mskel}^{\mathtt{V}}(t)) = (\lambda x.s, S)$ where $\mathtt{split}(\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(u)) = (s, S)$. Moreover, by definition $\mathtt{skdec}^{\mathtt{V}}(t) = (\lambda x.r, S')$ where $\mathtt{skdec}^{\mathtt{V}\cup\{x\}}(u) = (r, S')$. By *i.h.*, $(s, S) = (r, S')$, so that $(\lambda x.s, S) = (\lambda x.r, S')$.
   - **Case** $t = us$ **and** $\mathtt{fv}(t)\cap\mathtt{V} \neq \emptyset$**.** By definition:

     $$\mathtt{split}(\mathtt{mskel}^{\mathtt{V}}(t)) = \mathtt{split}(\mathtt{mskel}^{\mathtt{V}}(u)\circ\mathtt{mskel}^{\mathtt{V}}(s)) = (u's', S'\langle S\rangle)$$

     where $\mathtt{split}(\mathtt{mskel}^{\mathtt{V}}(u)) = (u', S)$ and $\mathtt{split}(\mathtt{mskel}^{\mathtt{V}}(s)) = (s', S')$. By *i.h.*, we have $\mathtt{skdec}^{\mathtt{V}}(u) = (u', S)$ and $\mathtt{skdec}^{\mathtt{V}}(s) = (s', S')$. Therefore, $\mathtt{skdec}^{\mathtt{V}}(t) = (u's', S'\langle S\rangle) = \mathtt{split}(\mathtt{mskel}^{\mathtt{V}}(t))$.
2. By definition, $\mathtt{skdec}(\lambda x.t) = (\lambda x.u, S)$ where $\mathtt{skdec}^{\{x\}}(t) = (u, S)$. By Point 1, we have $\mathtt{split}(\mathtt{mskel}^{\{x\}}(t)) = (u, S)$. Thus, $\mathtt{split}(\mathtt{mskel}(\lambda x.t)) = (\lambda x.u, S) = \mathtt{skdec}(\lambda x.t)$. □

Soundness of the Algorithm. First of all, we need an auxiliary lemma.

Lemma C.1. *Let t be a term, $\mathtt{V}$ a set of variables, and $y \notin \mathtt{fv}(t)$. Then $\mathtt{mskel}^{\mathtt{V}}(t) = \mathtt{mskel}^{\mathtt{V}\setminus\{y\}}(t)$.*

Proof. By induction on t.

- **Case t is such that $\mathtt{fv}(t)\cap\mathtt{V} = \emptyset$.** Then, we also have $\mathtt{fv}(t)\cap(\mathtt{V}\setminus\{y\}) = \emptyset$, so by definition $\mathtt{mskel}^{\mathtt{V}}(t) = t = \mathtt{mskel}^{\mathtt{V}\setminus\{y\}}(t)$.
- **Case $t = x$ and $x \in \mathtt{V}$.** By assumption $x \neq y$, hence by definition: $\mathtt{mskel}^{\mathtt{V}}(t) = x^{\circ} = \mathtt{mskel}^{\mathtt{V}\setminus\{y\}}(t)$.
- **Case $t = \lambda x.u$ and $\mathtt{fv}(t) \cap \mathtt{V} \neq \emptyset$.** Without loss of generality we can assume $x \neq y$. Thus, since by assumption $y \notin \mathtt{fv}(t)$, we also have that $y \notin \mathtt{fv}(u)$. Hence by *i.h.* we have:
$$\begin{aligned}\mathtt{mskel}^{\mathtt{V}}(t) &= \lambda^{\circ}x.\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(u)\\ &= \lambda^{\circ}x.\mathtt{mskel}^{(\mathtt{V}\cup\{x\})\setminus\{y\}}(u)\\ &= \lambda^{\circ}x.\mathtt{mskel}^{(\mathtt{V}\setminus\{y\})\cup\{x\}}(u)\\ &= \mathtt{mskel}^{\mathtt{V}\setminus\{y\}}(\lambda x.u)\\ &= \mathtt{mskel}^{\mathtt{V}\setminus\{y\}}(t).\end{aligned}$$
- **Case $t = us$ and $\mathtt{fv}(t) \cap \mathtt{V} \neq \emptyset$.** By assumption $y \notin \mathtt{fv}(t)$, therefore, $y \notin \mathtt{fv}(u)$ and $y \notin \mathtt{fv}(s)$. Furthermore, $\mathtt{fv}(t) \cap (\mathtt{V} \setminus \{y\}) \neq \emptyset$, so using the *i.h.*:
$$\begin{aligned}\mathtt{mskel}^{\mathtt{V}}(t) &= \mathtt{mskel}^{\mathtt{V}}(u) \circ \mathtt{mskel}^{\mathtt{V}}(s)\\ &= \mathtt{mskel}^{\mathtt{V}\setminus\{y\}}(u) \circ \mathtt{mskel}^{\mathtt{V}}(s)\\ &= \mathtt{mskel}^{\mathtt{V}\setminus\{y\}}(u) \circ \mathtt{mskel}^{\mathtt{V}\setminus\{y\}}(s)\\ &= \mathtt{mskel}^{\mathtt{V}\setminus\{y\}}(t).\end{aligned}$$
□

Lemma 5.7 (Crucial)**.** *Let t be a term, $\mathtt{V}$ a set of variables, $x \in \mathtt{fv}(t) \setminus \mathtt{V}$, $m := \mathtt{mskel}^{\mathtt{V}}(t)$, and $n := \mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t)$. Then:*

Originally at p. 13

$$m\{x\leftarrow\overline{x^{\circ}}^{\Uparrow}\} \to_{\mathtt{alg}}^{k} \begin{cases}\overline{n}^{\Uparrow} & \text{with } k = |n|_{\circ} - |m|_{\circ} - 1, \text{ if } m \text{ starts unmarked;}\\ n & \text{with } k = |n|_{\circ} - |m|_{\circ}, \text{ if } m \text{ starts with } \circ.\end{cases}$$

Proof. By induction on the structure of t.

- **Case $t = x$.** By definition, $m = x$ and $n = x^{\circ}$. Note that $m\{x\leftarrow\overline{x^{\circ}}^{\Uparrow}\} = x\{x\leftarrow\overline{x^{\circ}}^{\Uparrow}\} = \overline{x^{\circ}}^{\Uparrow}$, as required. Moreover, $k = 0$ and $|n|_{\circ} - |m|_{\circ} - 1 = 1 + 0 - 1 = 0$, as required.
- **Case $t = t_l t_r$ and $\mathtt{fv}(t) \cap \mathtt{V} = \emptyset$.** Now, observe that we also have that $\mathtt{fv}(t_l) \cap \mathtt{V} = \emptyset$ and $\mathtt{fv}(t_r) \cap \mathtt{V} = \emptyset$, therefore $\mathtt{mskel}^{\mathtt{V}}(t_l) = t_l$ and $\mathtt{mskel}^{\mathtt{V}}(t_r) = t_r$. Hence we can write $m = t_l t_r$. Concerning n, since $x \in \mathtt{fv}(t)$, we have:
$$n = \mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_l) \circ \mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_r).$$
Let $n_l = \mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_l)$ and $n_r = \mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_r)$. We distinguish three cases:

– **Case** $x \in \mathtt{fv}(t_l) \cap \mathtt{fv}(t_r)$**.** We have that:

$$\begin{array}{rlll} m\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & = & t_l\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\}t_r\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & \\ \text{(by } \textit{i.h.}) & \to_{\mathtt{alg}}^{k_1} & \overline{n_l}^{\Uparrow}t_r\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & \\ \text{(by } \textit{i.h.}) & \to_{\mathtt{alg}}^{k_2} & \overline{n_l}^{\Uparrow}\overline{n_r}^{\Uparrow} & \\ & \to_{\mathtt{pr1}} & \overline{(n_l \circ \overline{n_r}^{\Uparrow})}^{\Uparrow} & \\ & \to_{\mathtt{ab2}} & \overline{(n_l \circ n_r)}^{\Uparrow} & = \quad \overline{n}^{\Uparrow} \end{array}$$

where $k_1 =_{i.h.} |n_l|_\circ - |t_l|_\circ - 1 = |n_l|_\circ - 1$ and $k_2 =_{i.h.} |n_r|_\circ - |t_r|_\circ - 1 = |n_r|_\circ - 1$ because t is unmarked so that $|u_l|_\circ = |u_r|_\circ = 0$. Moreover:

$$\begin{array}{rlll} k & = & k_1 + k_2 + 2 & \\ & =_{i.h.} & |n_l|_\circ + |n_r|_\circ - 2 + 2 & \\ & = & |n_l|_\circ + |n_r|_\circ & \\ & = & |n|_\circ - 1 & = \quad |n|_\circ - |m|_\circ - 1 \end{array}$$

Then the statement holds, since m starts unmarked.

– **Case** $x \in \mathtt{fv}(t_l)$ **and** $x \notin \mathtt{fv}(t_r)$**.** Observe that $t_r\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} = t_r$, so we have that:

$$\begin{array}{rlll} m\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & = & t_l\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\}t_r\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & \\ & = & t_l\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\}t_r & \\ \text{(by } \textit{i.h.}) & \to_{\mathtt{alg}}^{k_1} & \overline{n_l}^{\Uparrow}t_r & \\ & \to_{\mathtt{alg}} & \overline{(n_l \circ t_r)}^{\Uparrow} & \\ & = & \overline{(n_l \circ \mathtt{mskel}^{\mathtt{V}}(t_r))}^{\Uparrow} & \\ & =_{L.C.1} & \overline{(n_l \circ \mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_r))}^{\Uparrow} & \\ & = & \overline{(n_l \circ n_r)}^{\Uparrow} & = \quad \overline{n}^{\Uparrow} \end{array}$$

where $k_1 =_{i.h.} |n_l|_\circ - |t_l|_\circ - 1 = |n_l|_\circ - 1$. Moreover:

$$\begin{array}{rlll} k & = & k_1 + 1 & \\ & =_{i.h.} & |n_l|_\circ - 1 + 1 & \\ & = & |n_l|_\circ & \\ & = & |n_l|_\circ + |n_r|_\circ & \\ & = & |n|_\circ - 1 & = \quad |n|_\circ - |m|_\circ - 1 \end{array}$$

Then the statement holds, since m starts unmarked.

– **Case** $x \notin \mathtt{fv}(t_l)$ **and** $x \in \mathtt{fv}(t_r)$**.** As in the previous case, simply switching the roles of t_l and t_r.

- **Case** $t = t_l t_r$ **and** $\mathtt{fv}(t) \cap \mathtt{V} \neq \emptyset$**.** By definition we have that $m = \mathtt{mskel}^{\mathtt{V}}(t_l) \circ \mathtt{mskel}^{\mathtt{V}}(t_r)$, that is, m starts with $\circ$. Since $\mathtt{fv}(t) \cap \mathtt{V} \cup \{x\} \neq \emptyset$, we also have that $n = \mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_l) \circ \mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_r)$. Now, let $m_l = \mathtt{mskel}^{\mathtt{V}}(t_l)$, $m_r = \mathtt{mskel}^{\mathtt{V}}(t_r)$, $n_l = \mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_l)$, $n_r = \mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_r)$. There are three cases.

  – **Case both** $\mathtt{fv}(t_l)$ **and** $\mathtt{fv}(t_r)$ **have non-empty intersection with** $\mathtt{V}$**.** Then both m_l and m_r start with $\circ$. Sub-cases:

* $x \in \mathtt{fv}(t_l) \cap \mathtt{fv}(t_r)$. Then:

$$\begin{array}{rlll} m\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & = & m_l\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} \circ m_r\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & \\ (\text{by } \textit{i.h.}) & \to^{k_1}_{\mathtt{alg}} & n_l \circ m_r\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & \\ (\text{by } \textit{i.h.}) & \to^{k_2}_{\mathtt{alg}} & n_l \circ n_r & = \quad n \end{array}$$

where $k_1 =_{i.h.} |n_l|_\circ - |m_l|_\circ$ and $k_2 =_{i.h.} |n_r|_\circ - |m_r|_\circ$. Moreover:

$$\begin{array}{rll} k & = & k_1 + k_2 \\ & =_{i.h.} & |n_l|_\circ - |m_l|_\circ + |n_r|_\circ - |m_r|_\circ \\ & = & |n_l|_\circ + |n_r|_\circ - |m_l|_\circ - |m_r|_\circ \\ & = & (|n_l|_\circ + |n_r|_\circ + 1) - (|m_l|_\circ + |m_r|_\circ + 1) \\ & = & |n|_\circ - |m|_\circ \end{array}$$

Then the statement holds, since m starts with $\circ$.

* $x \in \mathtt{fv}(t_l)$ *and* $x \notin \mathtt{fv}(t_r)$. By Lemma C.1, $x \notin \mathtt{fv}(t_r)$ implies $n_r = \mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_r) = \mathtt{mskel}^{\mathtt{V}}(t_r) = m_r$. Then:

$$\begin{array}{rlll} m\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & = & m_l\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} \circ m_r\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & \\ (\text{by } \textit{i.h.}) & \to^{k}_{\mathtt{alg}} & n_l \circ m_r\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & \\ & = & n_l \circ m_r & \\ & =_{L.C.1} & n_l \circ n_r & = \quad n \end{array}$$

where $k =_{i.h.} |n_l|_\circ - |m_l|_\circ$. Clearly, from $n_r = m_r$ it follows $|n_r|_\circ = |m_r|_\circ$ Moreover:

$$\begin{array}{rll} k & =_{i.h.} & |n_l|_\circ - |m_l|_\circ \\ & = & |n_l|_\circ + |n_r|_\circ - |m_l|_\circ - |m_r|_\circ \\ & = & (|n_l|_\circ + |n_r|_\circ + 1) - (|m_l|_\circ + |m_r|_\circ + 1) \\ & = & |n|_\circ - |m|_\circ \end{array}$$

Then the statement holds, since m starts with $\circ$.

* $x \notin \mathtt{fv}(t_l)$ *and* $x \in \mathtt{fv}(t_r)$. Specular to the previous one.

– **Case only $\mathtt{fv}(t_l)$ has non-empty intersection with $\mathtt{V}$.** Therefore, m_l starts with $\circ$ and $m_r = t_r$ starts unmarked.

* $x \in \mathtt{fv}(t_l) \cap \mathtt{fv}(t_r)$. Then:

$$\begin{array}{rlll} m\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & = & m_l\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} \circ t_r\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & \\ (\text{by } \textit{i.h.}) & \to^{k_1}_{\mathtt{alg}} & n_l \circ t_r\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & \\ (\text{by } \textit{i.h.}) & \to^{k_2}_{\mathtt{alg}} & n_l \circ \overline{n_r}^{\Uparrow} & \\ & \to_{\mathtt{ab2}} & n_l \circ n_r & = \quad n \end{array}$$

where $k_1 =_{i.h.} |n_l|_\circ - |m_l|_\circ$ and $k_2 =_{i.h.} |n_r|_\circ - |t_r|_\circ - 1 = |n_r|_\circ - 1$. Moreover:

$$\begin{array}{rlll} k & = & k_1 + k_2 + 1 & \\ & =_{i.h.} & |n_l|_\circ - |m_l|_\circ + |n_r|_\circ - 1 + 1 & \\ & = & |n_l|_\circ - |m_l|_\circ + |n_r|_\circ & \\ & = & (|n_l|_\circ + |n_r|_\circ + 1) - (|m_l|_\circ + 1) & = \quad |n|_\circ - |m|_\circ \end{array}$$

Then the statement holds, since m starts with $\circ$.

* $x \in \mathtt{fv}(t_l)$ *and* $x \notin \mathtt{fv}(t_r)$. By Lemma C.1, $x \notin \mathtt{fv}(t_r)$ implies $n_r = \mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_r) = \mathtt{mskel}^{\mathtt{V}}(t_r) = t_r$. Then:

$$\begin{array}{rlll} m\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & = & m_l\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} \circ t_r\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & \\ \text{(by } \textit{i.h.}) & \to^{k}_{\mathtt{alg}} & n_l \circ t_r\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & \\ & = & n_l \circ t_r & \\ & =_{L.C.1} & n_l \circ n_r & = \quad n \end{array}$$

where $k =_{i.h.} |n_l|_{\circ} - |m_l|_{\circ}$. Clearly, from $n_r = t_r$ it follows $|n_r|_{\circ} = |t_r|_{\circ} = 0$ Moreover:

$$\begin{array}{rll} k & =_{i.h.} & |n_l|_{\circ} - |m_l|_{\circ} \\ & = & |n_l|_{\circ} + |n_r|_{\circ} - |m_l|_{\circ} - |t_r|_{\circ} \\ & = & (|n_l|_{\circ} + |n_r|_{\circ} + 1) - (|m_l|_{\circ} + |t_r|_{\circ} + 1) \\ & = & |n|_{\circ} - |m|_{\circ} \end{array}$$

Then the statement holds, since m starts with $\circ$.

* $x \notin \mathtt{fv}(t_l)$ *and* $x \in \mathtt{fv}(t_r)$. By Lemma C.1, $x \notin \mathtt{fv}(t_l)$ implies $n_l = \mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_l) = \mathtt{mskel}^{\mathtt{V}}(t_l) = m_l$. Then:

$$\begin{array}{rlll} m\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & = & m_l\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} \circ t_r\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & \\ & = & m_l \circ t_r\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & \\ & =_{L.C.1} & n_l \circ t_r\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & \\ \text{(by } \textit{i.h.}) & \to^{k_r}_{\mathtt{alg}} & n_l \circ \overline{n_r}^{\Uparrow} & \\ & \to_{\mathtt{ab2}} & n_l \circ n_r & = \quad n \end{array}$$

where $k_r =_{i.h.} |n_r|_{\circ} - |t_r|_{\circ} - 1$. Clearly, from $n_l = m_l$ it follows $|n_l|_{\circ} = |m_l|_{\circ}$. Moreover:

$$\begin{array}{rll} k & = & k_r + 1 \\ & =_{i.h.} & |n_r|_{\circ} - |t_r|_{\circ} - 1 + 1 \\ & = & |n_r|_{\circ} - |t_r|_{\circ} \\ & = & |n_l|_{\circ} + |n_r|_{\circ} - |m_l|_{\circ} - |t_r|_{\circ} \\ & = & (|n_l|_{\circ} + |n_r|_{\circ} + 1) - (|m_l|_{\circ} + |t_r|_{\circ} + 1) \\ & = & |n|_{\circ} - |m|_{\circ} \end{array}$$

Then the statement holds, since m starts with $\circ$.

– **Case only** $\mathtt{fv}(t_r)$ **has non-empty intersection with** $\mathtt{V}$**.** As in the previous case, simply switching the roles of t_l and t_r.

- **Case** $t = \lambda y.t_b$ **and** $\mathtt{fv}(t) \cap \mathtt{V} = \emptyset$**.** Without loss of generality we can assume $y \notin \mathtt{V} \cup \{x\}$. By definition, $m = \lambda y.t_b$. Now, since $\mathtt{fv}(t) \cap \mathtt{V} = \emptyset$ and $y \notin \mathtt{V}$, we also have that $\mathtt{fv}(t_b) \cap \mathtt{V} = \emptyset$. Therefore, by definition we have that $t_b = \mathtt{mskel}^{\mathtt{V}}(t_b)$ and so we can write $m = \lambda y.\mathtt{mskel}^{\mathtt{V}}(t_b)$. Let $m_b := \mathtt{mskel}^{\mathtt{V}}(t_b)$. Observe that m_b starts unmarked and that $x \in \mathtt{fv}(t)$ implies $x \in \mathtt{fv}(t_b)$, so we have:

$$\begin{array}{rll} m\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\} & = & \lambda y.(m_b\{x{\leftarrow}\overline{x^{\circ}}^{\Uparrow}\}) \\ \text{(by } \textit{i.h.}) & \to^{k_1}_{\mathtt{alg}} & \lambda y.(\overline{\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_b)}^{\Uparrow}) \\ & \to_{\mathtt{pr3}} & \overline{(\lambda^{\circ} y.\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_b)\{y{\leftarrow}\overline{y^{\circ}}^{\Uparrow}\})}^{\Uparrow} \end{array}$$

where $k_1 =_{i.h.} |\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_b)|_\circ - |m_b|_\circ - 1 = |\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_b)|_\circ - 1$. Now, we distinguish two cases.

- **Case** $y \in \mathtt{fv}(t_b)$**.** By definition, $|\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_b)|_\circ$ starts with $\circ$. Therefore, applying the *i.h.* again:

$$\begin{array}{rlll} m\{x{\leftarrow}\overline{x^\circ}^{\Uparrow}\} & \to_{\mathtt{alg}}^{k_1+1} & \overline{(\lambda^\circ y.\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_b)\{y{\leftarrow}\overline{y^\circ}^{\Uparrow}\})}^{\Uparrow} & \\ (\text{by } \textit{i.h.}) & \to_{\mathtt{alg}}^{k_2} & \overline{(\lambda^\circ y.\mathtt{mskel}^{\mathtt{V}\cup\{x\}\cup\{y\})}(t_b))}^{\Uparrow} & \\ & = & \overline{\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(\lambda y.t_b)}^{\Uparrow} & = \quad \overline{n}^{\Uparrow} \end{array}$$

  where $k_2 =_{i.h.} |\mathtt{mskel}^{\mathtt{V}\cup\{x\}\cup\{y\})}(t_b)|_\circ - |\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_b)|_\circ$. Since also $\mathtt{mskel}^{\mathtt{V}\cup\{x\}\cup\{y\}}(t_b)$ starts with $\circ$, we have (remember that $|m|_\circ = 0$):

$$\begin{array}{rll} k & = & k_1 + 1 + k_2 \\ & =_{i.h.} & |\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_b)|_\circ - 1 + 1 + |\mathtt{mskel}^{\mathtt{V}\cup\{x\}\cup\{y\})}(t_b)|_\circ - |\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_b)|_\circ \\ & = & |\mathtt{mskel}^{\mathtt{V}\cup\{x\}\cup\{y\})}(t_b)|_\circ \\ & = & |n|_\circ - 1 \\ & = & |n|_\circ - |m|_\circ - 1. \end{array}$$

  Then the statement holds, since m starts unmarked.

- **Case** $y \notin \mathtt{fv}(t_b)$**.** Then, $\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_b)\{y{\leftarrow}\overline{y^\circ}^{\Uparrow}\} = \mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_b)$. By Lemma C.1, since $y \notin \mathtt{fv}(t_b)$ we also have that $\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_b) = \mathtt{mskel}^{\mathtt{V}\cup\{x\}\cup\{y\}}(t_b)$. Therefore:

$$\begin{array}{rlll} m\{x{\leftarrow}\overline{x^\circ}^{\Uparrow}\} & \to_{\mathtt{alg}}^{k_1+1} & \overline{(\lambda^\circ y.\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_b)\{y{\leftarrow}\overline{y^\circ}^{\Uparrow}\})}^{\Uparrow} & \\ & = & \overline{(\lambda^\circ y.\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_b))}^{\Uparrow} & \\ & =_{L.C.1} & \overline{(\lambda^\circ y.\mathtt{mskel}^{\mathtt{V}\cup\{x\}\cup\{y\}}(t_b))}^{\Uparrow} & \\ & = & \overline{\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(\lambda y.t_b))}^{\Uparrow} & = \quad \overline{n}^{\Uparrow} \end{array}$$

  With:

$$\begin{array}{rll} k & = & k_1 + 1 \\ & =_{i.h.} & |\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_b)|_\circ - 1 + 1 \\ & = & |\mathtt{mskel}^{\mathtt{V}\cup\{x\}}(t_b)|_\circ \\ & = & |\mathtt{mskel}^{\mathtt{V}\cup\{x\}\cup\{y\}}(t_b)|_\circ \\ & = & |\lambda^\circ y.\mathtt{mskel}^{\mathtt{V}\cup\{x\}\cup\{y\}}(t_b)|_\circ - 1 \\ & = & |n|_\circ - 1 \qquad = \quad |n|_\circ - |m|_\circ - 1. \end{array}$$

  Then the statement holds, since m starts unmarked.

- **Case** $t = \lambda y.t_b$ **and** $\mathtt{fv}(t) \cap \mathtt{V} \neq \emptyset$**.** Without loss of generality we can assume $y \notin \mathtt{V} \cup \{x\}$. By definition, $m = \lambda^\circ y.\mathtt{mskel}^{\mathtt{V}\cup\{y\}}(t_b)$, that is, m starts with $\circ$. Let $m_b = \mathtt{mskel}^{\mathtt{V}\cup\{y\}}(t_b)$. Note that $\mathtt{fv}(t) \cap \mathtt{V} \neq \emptyset$ implies $\mathtt{fv}(t_b) \cap \mathtt{V} \neq \emptyset$, in turn implying that m_b starts with $\circ$. Then:

$$\begin{array}{rlll} m\{x{\leftarrow}\overline{x^\circ}^{\Uparrow}\} & = & \lambda^\circ y.m_b\{x{\leftarrow}\overline{x^\circ}^{\Uparrow}\} & \\ (\text{by } \textit{i.h.}) & \to_{\mathtt{alg}}^{k} & \lambda^\circ y.\mathtt{mskel}^{\mathtt{V}\cup\{x\}\cup\{y\}}(t_b) & \\ & = & \mathtt{mskel}^{\mathtt{V}\cup\{x\}}(\lambda y.t_b) & = \quad n. \end{array}$$

Where $k =_{i.h.} |\mathtt{mskel}^{\mathtt{V}\cup\{x\}\cup\{y\}}(t_b)|_\circ - |m_b|_\circ$. Then, since both m and n start with $\circ$:

$$\begin{array}{rcl} k & =_{i.h.} & |\mathtt{mskel}^{\mathtt{V}\cup\{x\}\cup\{y\}}(t_b)|_\circ - |m_b|_\circ \\ & = & |\mathtt{mskel}^{\mathtt{V}\cup\{x\}\cup\{y\}}(t_b)|_\circ + 1 - |m_b|_\circ - 1 \quad = \quad |n|_\circ - |m|_\circ \quad \square \end{array}$$

D Proofs omitted from Sect. 6 (Preliminaries about Abstract Machines)

Lemma D.1 (One-step simulation)**.** *Let a machine* $\mathtt{M}$ *and a structural strategy* $(\to_{\mathtt{str}}, \equiv)$ *be a distillery. For any state* $\mathtt{Q}$ *of* $\mathtt{M}$*, if* $\underline{\mathtt{Q}} \to_{\mathtt{str}} \equiv u$ *then there is a state* $\mathtt{Q}'$ *of* $\mathtt{M}$ *such that* $\mathtt{Q} \rightsquigarrow^*_{\mathtt{sea}} \rightsquigarrow_{\mathtt{pr}} \mathtt{Q}'$ *and* $\underline{\mathtt{Q}'} \equiv u$ *and such that the label of the principal transition and of the step are the same.*

Proof. For any state $\mathtt{Q}$ of $\mathtt{M}$, let $\mathtt{nf}_{\mathtt{sea}}(\mathtt{Q})$ be a normal form of $\mathtt{Q}$ with respect to $\rightsquigarrow_{\mathtt{sea}}$: such a state exists because search transitions terminate (Point 3 of Definition 6.2). Since $\rightsquigarrow_{\mathtt{sea}}$ is mapped on identities (by *search transparency*, *i.e.* Point 2 of Definition 6.2), one has $\underline{\mathtt{nf}_{\mathtt{sea}}(\mathtt{Q})} = \underline{\mathtt{Q}}$. Since $\underline{\mathtt{Q}}$ is not $\to_{\mathtt{str}}$-normal by hypothesis, the halt property (Point 5) entails that $\mathtt{nf}_{\mathtt{sea}}(\mathtt{Q})$ is not final, therefore $\mathtt{Q} \rightsquigarrow^*_{\mathtt{sea}} \mathtt{nf}_{\mathtt{sea}}(\mathtt{Q}) \rightsquigarrow_{\mathtt{pr}} \mathtt{Q}'$ for some state $\mathtt{Q}'$, and thus $\underline{\mathtt{Q}} = \underline{\mathtt{nf}_{\mathtt{sea}}(\mathtt{Q})} \to_{\mathtt{str}} \equiv \underline{\mathtt{Q}'}$ by principal projection (Point 1). By determinism of $\to_{\mathtt{str}}$ (Point 4), one obtains $\underline{\mathtt{Q}'} \equiv u$. $\square$

Theorem 6.3 (Sufficient condition for implementations)**.** *Let a machine* $\mathtt{M}$ *and a structural strategy* $(\to_{\mathtt{str}}, \equiv)$ *be a distillery. Then, they are mechanical bisimilar.*

Originally at p. 15

Proof. According to Definition 6.1, given a $\Lambda_{\mathtt{sk}}$-term t and an initial state $t \triangleleft \mathtt{Q}$, we have to show that:

1. *Runs to evaluations*: for any $\mathtt{M}$-run $r : t \triangleleft \mathtt{Q} \rightsquigarrow^*_{\mathtt{M}} \mathtt{Q}'$ there exists a $\to_{\mathtt{str}}$-evaluation $\mathtt{e} : t \to^*_{\mathtt{str}} \equiv \underline{\mathtt{Q}'}$;
2. *Evaluations to runs*: for every $\to_{\mathtt{str}}$-evaluation $\mathtt{e} : t \to^*_{\mathtt{str}} u$ there exists a $\mathtt{M}$-run $r : t \triangleleft \mathtt{Q} \rightsquigarrow^*_{\mathtt{M}} \mathtt{Q}'$ such that $\underline{\mathtt{Q}'} \equiv u$;

Plus the principal matching constraint that shall be evident by the principal projection requirement and how the proof is built.

Proof of Point 1. By induction on $|r|_{\mathtt{pr}} \in \mathbb{N}$. Cases:

- $|r|_{\mathtt{pr}} = 0$. Then $r\colon t \triangleleft \mathtt{Q} \rightsquigarrow^*_{\mathtt{sea}} \mathtt{Q}'$ and hence $\underline{\mathtt{Q}} = \underline{\mathtt{Q}'}$ by search transparency (Point 2 of Definition 6.2). Moreover, $t = \underline{\mathtt{Q}}$ since decoding is the inverse of initialization, therefore the statement holds with respect to the empty evaluation $\mathtt{e}$ with starting and end term t.
- $|r|_{\mathtt{pr}} > 0$. Then, $r\colon t \triangleleft \mathtt{Q} \rightsquigarrow^*_{\mathtt{M}} \mathtt{Q}'$ is the concatenation of a run $r'\colon t \triangleleft \mathtt{Q} \rightsquigarrow^*_{\mathtt{M}} \mathtt{Q}''$ followed by a run $r''\colon \mathtt{Q}'' \rightsquigarrow_{\mathtt{pr}} \mathtt{Q}''' \rightsquigarrow^*_{\mathtt{sea}} \mathtt{Q}'$. By *i.h.* applied to r', there exists an evaluation $\mathtt{e}'\colon t \to^*_{\mathtt{str}} \equiv \underline{\mathtt{Q}'}$ satisfying the principal matching constraint. By principal projection (Point 1 of Definition 6.2) and search

transparency (Point 2 of Definition 6.2) applied to r'', one obtains a one-step evaluation $\mathtt{e}'' \colon \underline{\mathtt{Q}'} \to_{\mathtt{str}} \equiv \underline{\mathtt{Q}''} = \underline{\mathtt{Q}}$ having the same principal label of the transition. Concatenating $\mathtt{e}'$ and $\mathtt{e}''$, we obtain an evaluation:

$$\mathtt{e}''' \colon t \to^*_{\mathtt{str}} \equiv \underline{\mathtt{Q}'} \to_{\mathtt{str}} \equiv \underline{\mathtt{Q}}.$$

Postponing structural equivalence (Prop. 3.4) we obtain the required evaluation $\mathtt{e} \colon t \to^*_{\mathtt{str}} \equiv \underline{\mathtt{Q}}$, which satisfies the principal matching constraint.

Proof of Point 2. By induction on $|\mathtt{e}| \in \mathbb{N}$. Cases:

- $|\mathtt{e}| = 0$. Then $t = u$. Since decoding is the inverse of initialization, one has $\underline{\mathtt{Q}} = t$. Then the statement holds with respect to the empty (*i.e.* without transitions) run r with initial (and final) state $\mathtt{Q}$.
- $|\mathtt{e}| > 0$. Then, $\mathtt{e} \colon t \to^*_{\mathtt{str}} u$ is the concatenation of an evaluation $\mathtt{e}' \colon t \to^*_{\mathtt{str}} u'$ followed by the step $u' \to_{\mathtt{str}} u$. By *i.h.*, there exists a $\mathtt{M}$-run $r' \colon t \triangleleft \mathtt{Q} \rightsquigarrow^*_{\mathtt{M}} \mathtt{Q}''$ such that $\underline{\mathtt{Q}''} \equiv u'$ and verifying the principal matching constraint. Since $\underline{\mathtt{Q}''} \equiv u' \to_{\mathtt{str}} u$, by strong bisimulation of $\equiv$ (Prop. 3.4) we obtain $\underline{\mathtt{Q}''} \to_{\mathtt{str}} \equiv u$. By one-step simulation (Lemma D.1), there is a state $\mathtt{Q}'$ of $\mathtt{M}$ such that $\mathtt{Q}'' \rightsquigarrow^*_{\mathtt{sea}} \rightsquigarrow_{\mathtt{pr}} \mathtt{Q}'$ and $\underline{\mathtt{Q}'} \equiv u$, preserving the label of the step/transition. Therefore, the run $r \colon t \triangleleft \mathtt{Q} \rightsquigarrow^*_{\mathtt{M}} \mathtt{Q}'' \rightsquigarrow^*_{\mathtt{sea}} \rightsquigarrow_{\mathtt{pr}} \mathtt{Q}'$ satisfies the statement. □

E Proofs omitted from Sect. 7 (The MAD and the Skeletal MAD)

Lemma 7.2 (Qualitative invariants)**.** *Let $t \triangleleft \mathtt{Q} \rightsquigarrow^* \mathtt{Q}' = (\mathtt{C}, u, \mathtt{S}, \mathtt{E})$ be a Skeletal MAD run.* Originally at p. 17

1. Closure*: for every term s of $\mathtt{Q}'$ and for any variable $x \in \mathtt{fv}(s)$ there is an environment entry $[x{\leftarrow}t]$, $\langle x{\leftarrow}v\rangle$, or $[x{\leftarrow}\cdot]$ on the right of s in $\mathtt{Q}'$.*
2. Well-bound*: if $\lambda x.s$ occurs in $\mathtt{Q}'$ and x has any other occurrence in $\mathtt{Q}'$ then it is as a free variable of s, and for any substitution $[y{\leftarrow}t]$, $\langle y{\leftarrow}v\rangle$, or $[y{\leftarrow}\cdot]$ in $\mathtt{Q}'$ the name y can occur (in any form) only on the left of that entry in $\mathtt{Q}'$.*
3. Contextual chain read-back*: $\underline{\mathtt{C}}$, $\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\rangle$, $\underline{\mathtt{E}}\langle\underline{\mathtt{C}}\rangle$ and $\underline{\mathtt{E}}\langle\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\rangle\rangle$ are CbNeed evaluation contexts.*

Proof. By induction on the length of the run. The base case trivially holds, the inductive case is by analysis of the last transition. The first two invariants are proved by straightforward inspection of the transitions. For the third one, the crucial point is that the well-bound invariant ensures that in a chain $\mathtt{C}{:}(x, \mathtt{S}, \mathtt{E}[x{\leftarrow}\cdot])$ there are no environment entries of variable x in $\mathtt{E}$ or $\mathtt{C}$, so that in the definition of read-back $\underline{\mathtt{C}{:}(x, \mathtt{S}, \mathtt{E}[x{\leftarrow}\cdot])} := \underline{\mathtt{E}}\langle\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\langle x\rangle\rangle\rangle[x{\leftarrow}\langle\cdot\rangle]$ the context $\underline{\mathtt{E}}\langle\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\rangle\rangle$ does not bind x and it is indeed $[x{\leftarrow}\langle\cdot\rangle]$ that binds it. □

Theorem 7.3 (Skeletal distillery)**.** *The Skeletal MAD and the structural strategy $(\to_{\mathtt{sk{\cdot}need}}, \equiv)$ form a distillery. Namely, let $\mathtt{Q}$ be a reachable Skeletal MAD state.* Originally at p. 18

1. Principal projection*:*

(a) if $\mathtt{Q} \rightsquigarrow_{\beta} \mathtt{Q}'$ *then* $\underline{\mathtt{Q}} \rightarrow_{\mathtt{sk{\cdot}dB}} \equiv \underline{\mathtt{Q}'}$.

(b) if $\mathtt{Q} \rightsquigarrow_{\mathtt{sk}} \mathtt{Q}'$ *then* $\underline{\mathtt{Q}} \rightarrow_{\mathtt{sk}} \underline{\mathtt{Q}'}$.

(c) if $\mathtt{Q} \rightsquigarrow_{\mathtt{ss}} \mathtt{Q}'$ *then* $\underline{\mathtt{Q}} \rightarrow_{\mathtt{ss}} \underline{\mathtt{Q}'}$.

2. Search transparency*: if* $\mathtt{Q} \rightsquigarrow_{\mathtt{sea}_1,\mathtt{sea}_2,\mathtt{sea}_3} \mathtt{Q}'$ *then* $\underline{\mathtt{Q}} = \underline{\mathtt{Q}'}$.

3. Search terminates*:* $\rightsquigarrow_{\mathtt{sea}_1,\mathtt{sea}_2,\mathtt{sea}_3}$ *is terminating.*

4. Halt*: if* $\mathtt{Q}$ *is final then it has shape* $(\epsilon, v, \epsilon, \mathtt{E})$ *and* $\underline{\mathtt{Q}}$ *is* $\rightarrow_{\mathtt{sk{\cdot}need}}$*-normal.*

Proof.

1. (a) If $\mathtt{Q} = (\mathtt{C}, \lambda x.t, u{:}\mathtt{S}, \mathtt{E}) \rightsquigarrow_{\beta} (\mathtt{C}, t, \mathtt{S}, [x{\leftarrow}u]{:}\mathtt{E}) = \mathtt{Q}'$ then:

$$\begin{array}{rlll} \underline{(\mathtt{C}, \lambda x.t, u{:}\mathtt{S}, \mathtt{E})} & = & \underline{\mathtt{E}}\langle\underline{\mathtt{C}}\langle\underline{u{:}\mathtt{S}}\langle\lambda x.t\rangle\rangle\rangle & \\ & = & \underline{\mathtt{E}}\langle\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\langle(\lambda x.t)u\rangle\rangle\rangle & \\ (L.7.2.3) & \rightarrow_{\mathtt{sk{\cdot}dB}} & \underline{\mathtt{E}}\langle\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\langle t[x{\leftarrow}u]\rangle\rangle\rangle & \\ & \equiv_{L.7.2.3} & \underline{\mathtt{E}}\langle\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\langle t\rangle\rangle[x{\leftarrow}u]\rangle & \\ & = & \underline{[x{\leftarrow}u]{:}\mathtt{E}}\langle\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\langle t\rangle\rangle\rangle & = \quad \underline{(\mathtt{C}, t, \mathtt{S}, [x{\leftarrow}u]{:}\mathtt{E})}. \end{array}$$

Note that the $\mathtt{dB}$ steps applies because by the contextual chain read-back invariant (L.7.2.3) $(\lambda x.t)u$ occurs inside an evaluation context. Because of the same invariant, we can apply structural equivalence.

(b) If $\mathtt{Q} = (\mathtt{C}, x, \mathtt{S}, \mathtt{E}{:}[x{\leftarrow}v]{:}\mathtt{E}') \rightsquigarrow_{\mathtt{sk}} (\mathtt{C}, x, \mathtt{S}, \mathtt{E}{:}\langle x{\leftarrow}v'\rangle{:}\mathtt{E}_S{:}\mathtt{E}') = \mathtt{Q}'$ then:

$$\begin{array}{rlll} \underline{(\mathtt{C}, x, \mathtt{S}, \mathtt{E}{:}[x{\leftarrow}v]{:}\mathtt{E}')} & = & \underline{\mathtt{E}{:}[x{\leftarrow}v]{:}\mathtt{E}'}\langle\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\langle x\rangle\rangle\rangle & \\ & = & \underline{\mathtt{E}'}\langle\underline{\mathtt{E}}\langle\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\langle x\rangle\rangle\rangle[x{\leftarrow}v]\rangle & \\ (L.7.2.2, 3) & \rightarrow_{\mathtt{sk}} & \underline{\mathtt{E}'}\langle S\langle\underline{\mathtt{E}}\langle\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\langle x\rangle\rangle\rangle\langle x{\leftarrow}v'\rangle\rangle\rangle & \\ & = & \underline{\mathtt{E}{:}\mathtt{E}_S{:}\langle x{\leftarrow}v'\rangle{:}\mathtt{E}'}\langle\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\langle x\rangle\rangle\rangle & = \quad \underline{(\mathtt{C}, x, \mathtt{S}, \mathtt{E}{:}\langle x{\leftarrow}v'\rangle{:}\mathtt{E}_S{:}\mathtt{E}')}. \end{array}$$

The $\mathtt{sk}$ steps applies because:

i. As in the previous case, the contextual chain read-back invariant (L.7.2.3) ensures that the step takes place in an evaluation context;

ii. The well-bound invariant (Lemma 7.2.2) ensures that there is only one environment entry on x in $\mathtt{Q}$, so that there are no entries bounding x in $\mathtt{E}$ (if there were another entry $[x{\leftarrow}t]$ in $\mathtt{E}$ then the occurrence of x in $[x{\leftarrow}v]$ would violate the invariant).

(c) The case $\rightsquigarrow_{\mathtt{ss}}$ is analogous to the $\mathtt{sk}$ case.

2. Cases of the search transition:

(a) If $\mathtt{Q} = (\mathtt{C}, tu, \mathtt{S}, \mathtt{E}) \rightsquigarrow_{\mathtt{sea}_1} (\mathtt{C}, t, u{:}\mathtt{S}, \mathtt{E}) = \mathtt{Q}'$ then:

$$\underline{(\mathtt{C}, tu, \mathtt{S}, \mathtt{E})} \quad = \quad \underline{\mathtt{E}}\langle\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\langle tu\rangle\rangle\rangle \quad = \quad \underline{\mathtt{E}}\langle\underline{\mathtt{C}}\langle\underline{u{:}\mathtt{S}}\langle t\rangle\rangle\rangle \quad = \quad \underline{(\mathtt{C}, t, u{:}\mathtt{S}, \mathtt{E})}.$$

(b) If $\mathtt{Q} = (\mathtt{C}, x, \mathtt{S}, \mathtt{E}{:}[x{\leftarrow}t]{:}\mathtt{E}') \rightsquigarrow_{\mathtt{sea}_2} (\mathtt{C}{:}(x, \mathtt{S}, \mathtt{E}[x{\leftarrow}\cdot]), t, \epsilon, \mathtt{E}') = \mathtt{Q}'$ then:

$$\begin{array}{rlll} \underline{(\mathtt{C}, x, \mathtt{S}, \mathtt{E}{:}[x{\leftarrow}t]{:}\mathtt{E}')} & = & \underline{\mathtt{E}{:}[x{\leftarrow}t]{:}\mathtt{E}'}\langle\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\langle x\rangle\rangle\rangle & \\ & = & \underline{\mathtt{E}'}\langle\underline{\mathtt{E}}\langle\underline{\mathtt{C}}\langle\underline{\mathtt{S}}\langle x\rangle\rangle\rangle[x{\leftarrow}t]\rangle & = \quad \underline{(\mathtt{C}{:}(x, \mathtt{S}, \mathtt{E}[x{\leftarrow}\cdot]), t, \epsilon, \mathtt{E}')}. \end{array}$$

(c) If $\mathtt{Q} = (\mathtt{C}{:}(x, \mathtt{S}, \mathtt{E}[x\leftarrow\cdot]), v, \epsilon, \mathtt{E}') \rightsquigarrow_{\mathtt{sea}_3} (\mathtt{C}, x, \mathtt{S}, \mathtt{E}{:}[x\leftarrow v]{:}\mathtt{E}') = \mathtt{Q}'$ then:

$$\begin{array}{rcl} \underline{(\mathtt{C}{:}(x, \mathtt{S}, \mathtt{E}[x\leftarrow\cdot]), v, \epsilon, \mathtt{E}')} & = & \underline{\mathtt{E}'\langle\mathtt{E}\langle\mathtt{C}\langle\mathtt{S}\langle x\rangle\rangle\rangle[x\leftarrow v]\rangle} \\ & = & \underline{\mathtt{E}{:}[x\leftarrow v]{:}\mathtt{E}'}\langle\mathtt{C}\langle\mathtt{S}\langle x\rangle\rangle\rangle \quad = \quad \underline{(\mathtt{C}, x, \mathtt{S}, \mathtt{E}{:}[x\leftarrow v]{:}\mathtt{E}')}. \end{array}$$

3. Termination is implied by the complexity analysis of the next section.

4. Let us first show that final states $\mathtt{Q}_f$ have shape $(\epsilon, v, \epsilon, \mathtt{E})$. We have the following points:

   - *The code of a final state is an abstraction*: it cannot be an application, because transition $\mathtt{sea}_1$ would apply, and cannot be a variable x, because by the closure invariant (Lemma 7.2.1) there has to be an environment entry on x, and so one among transitions $\mathtt{sea}_2$, $\mathtt{sk}$, and $\mathtt{ss}$ would apply;
   - *The stack is empty*: because, given that the code is an abstraction, is the stack is non-empty then a β transition would apply;
   - *The chain is empty*: because, given that the code is an abstraction and the stack is empty, then transition $\mathtt{sea}_3$ would apply.

   Now, final states decode to terms of shape $S\langle v\rangle$ which are $\rightarrow_{\mathtt{sk}\cdot\mathtt{need}}$ normal. □

F Proofs Omitted from Sect. 8 (Complexity Analysis)

Lemma 8.1 (Skeleton and flesh size)**.** Originally at p. 18

1. Auxiliary parametrized statement*: let $t \in \Lambda$, $\mathtt{V}$ be a set of variables, and $\mathtt{skdec}^{\mathtt{V}}(t) := (u, S)$. Then $|u| \leq |t|$ and $|s| \leq |t|$ for any ES $[x\leftarrow s]$ in S;*
2. Main*: let $v \in \Lambda$. Then $|\mathtt{skel}(v)| \leq |v|$ and $|s| \leq |v|$ for any ES $[x\leftarrow s]$ in $\mathtt{flesh}(v)$.*

Proof. Point 1 is by induction on $\mathtt{skdec}^{\mathtt{V}}(t)$. Point 2 follows from Point 1 and the definitions of skeleton and flesh. □

Lemma 8.2 (Quantitative sub-term property)**.** *Let $t \triangleleft \mathtt{Q} \rightsquigarrow^*_{\mathtt{SMAD}} \mathtt{Q}'$ be a Skeletal MAD run. Then $|u| \leq |t|$ for any term u of $\mathtt{Q}'$.* Originally at p. 18

Proof. By induction on the length of the run. The base case trivially holds, the inductive case is by analysis of the last transition. For all transitions but $\rightsquigarrow_{\mathtt{sk}}$ it follows from the *i.h.* and a straightforward inspection of the transitions. For $\rightsquigarrow_{\mathtt{sk}}$, it is given by Lemma 8.1.2. □

G OCaml Implementation

We provide at `https://github.com/Franciman/fullylazymad` a reference implementation in OCaml of the Skeletal MAD, meant to guarantee the soundness of the assumptions that underly the complexity analyses. The implementation takes in input a user provided term and it shows the full run printing all transitions and all intermediate machine states. We highlight here a few details on the implementation.

Data structures. Pre-terms are represented by the following algebraic data type (ADT):

```
type term =
  | Var of { v: var_ref; mutable taken: bool; mutable parent: term option }
  | Abs of { v: var_ref; mutable body: term; mutable occurrences: term list; mutable taken:
     bool; mutable parent: term option }
  | App of { mutable head: term; mutable arg: term; mutable taken: bool; mutable parent: term
     option }
and var_ref =
 { mutable prev : var_ref option;
   orig_name : string; (* to help generating new names *)
   name : string;
   mutable sub : sub;
   mutable next : var_ref option }
and sub =
  NoSub | Sub of term | SubSkel of term | Hole | Copy of (term list ref * var_ref)
```

whose constructors `App` for applications, `Abs` for abstractions and `Var` for variable occurrence hold mutable pointers to their subterms, a mutable optional pointer to the parent and a mutable boolean `taken` for marking terms during skeleton extraction.

Variable occurrences point to a shared, unique variable declaration/substitution in memory of type `var_ref`, which holds the variable name, a substitution description field of type `sub` and two optional pointers (`prev` and `next`) to insert the variable in an environment, that is a double-linked lists of variable declarations. A substitution description is an ADT whose constructors are `Sub` t for the explicit substitution $[x{\leftarrow}t]$, `SubSkel` t for the skeletal substitutions $\langle x{\leftarrow}v\rangle$, `NoSub` for bound variables, `Hole` to represent $[x{\leftarrow}\cdot]$ in chain items and `Copy (parents,`x`)` to point to another variable declaration x together with the list of its parents. The latter form is used temporarily to preserve sharing of variable declarations during the implementation of $\cdot^{\alpha}$: the first time a variable is to be copied its substitution is make to point to the new copy and each time an occurrence of the variable is to be copied, the same variable declaration is reused and a new parent is added for it.

Abstractions also point to the list of occurrences of the bound variable, to implement rule $\mapsto_{\mathtt{pr3}}$ in $\mathcal{O}(1)$. This is a minor memory-saving divergence from the paper where it was suggested to have backpointers to the occurrences stored in the bound variable declaration node, of type `var_ref`: once an abstraction is consumed, the backpointers become useless and thus avoiding them in the implementation saves some space.

Machine states are implemented straightforwardly using lists for stacks, chains, and the set of roots (which is then actually represented as a list, not as a set), and doubly linked lists of variable declarations for environments:

```
type env = var_ref option
  (* Some head of the doubly-linked list, or None for empty list *)
type stack = term list
type chain = (var_ref * stack * env) list
type state = chain * term * stack * env
```

Skeleton Reconstruction. The rewriting system for marking terms to extract the skeleton and flesh is implemented sequentially as a recursive function. Each activation record for a call to `mark_skeleton` in the OCaml stack corresponds in the paper to an occurrence of $\cdot{:}\Uparrow$ in the rewritten term.

```
let rec mark_skeleton =
 function
```

```
  | None -> ()
  | Some (Var v) -> (v.taken <- true; mark_skeleton v.parent)
  | Some (Abs a) ->
      if not a.taken
      then (a.taken <- true;
            List.iter (fun v -> mark_skeleton (Some v)) a.occurrences;
            mark_skeleton a.parent)
  | Some (App a) ->
      if not a.taken
      then (a.taken <- true; mark_skeleton a.parent)
```

Parallel implementations of `mark_skeleton` are possible, but because of the necessity of syncronization over application nodes and because the size of skeletonized terms is supposed to be small, it is unclear if actual parallelization would provide any speed-up.

Remaining Code. The code to extract the skeleton and flesh from marked terms, and the code for machine transitions follow straightforwardly the rules in the paper, up to noisy code to keep the doubly linked structure of terms and environments. Here is an example of the code for one transition in the Skeletal MAD:

```
let step : state -> string * state =
 function
  ...
  | chain, Abs { v; body; _ }, arg :: args, env ->
      set_parent body None;  (* unliks body from its parent (double link severed)*)
      v.sub <- Sub arg;      (* turns v into an explicit substitution *)
      push v env;            (* pushes v on top of the env *)
      "β",(chain, body, args, Some v)
  ...
```

The code that implements $\cdot^{\alpha}$ also follows the standard algorithm to copy a graph in linear time preserving the sharing, described by Accattoli and Barras in [5]. It uses the `Copy` constructor to temporarily associate each variable to its copy.

References

[1] Beniamino Accattoli. An abstract factorization theorem for explicit substitutions. In Ashish Tiwari, editor, *23rd International Conference on Rewriting Techniques and Applications (RTA'12) , RTA 2012, May 28 - June 2, 2012, Nagoya, Japan*, volume 15 of *LIPIcs*, pages 6–21. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2012. doi:10.4230/LIPICS.RTA.2012.6.

[2] Beniamino Accattoli. Proof nets and the call-by-value λ-calculus. *Theor. Comput. Sci.*, 606:2–24, 2015. doi:10.1016/J.TCS.2015.08.006.

[3] Beniamino Accattoli. Proof nets and the linear substitution calculus. In Bernd Fischer and Tarmo Uustalu, editors, *Theoretical Aspects of Computing - ICTAC 2018 - 15th International Colloquium, Stellenbosch, South Africa, October 16-19, 2018, Proceedings*, volume 11187 of *Lecture Notes in Computer Science*, pages 37–61. Springer, 2018. doi:10.1007/978-3-030-02508-3_3.

[4] Beniamino Accattoli, Pablo Barenbaum, and Damiano Mazza. Distilling abstract machines. In Johan Jeuring and Manuel M. T. Chakravarty, editors, *Proceedings of the 19th ACM SIGPLAN international conference on*

Functional programming, Gothenburg, Sweden, September 1-3, 2014, pages 363–376. ACM, 2014. doi:10.1145/2628136.2628154.

[5] Beniamino Accattoli and Bruno Barras. Environments and the complexity of abstract machines. In Wim Vanhoof and Brigitte Pientka, editors, *Proceedings of the 19th International Symposium on Principles and Practice of Declarative Programming, Namur, Belgium, October 09 - 11, 2017*, pages 4–16. ACM, 2017. doi:10.1145/3131851.3131855.

[6] Beniamino Accattoli, Eduardo Bonelli, Delia Kesner, and Carlos Lombardi. A nonstandard standardization theorem. In Suresh Jagannathan and Peter Sewell, editors, *The 41st Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '14, San Diego, CA, USA, January 20-21, 2014*, pages 659–670. ACM, 2014. doi:10.1145/2535838.2535886.

[7] Beniamino Accattoli, Andrea Condoluci, Giulio Guerrieri, and Claudio Sacerdoti Coen. Crumbling abstract machines. In Ekaterina Komendantskaya, editor, *Proceedings of the 21st International Symposium on Principles and Practice of Programming Languages, PPDP 2019, Porto, Portugal, October 7-9, 2019*, pages 4:1–4:15. ACM, 2019. doi:10.1145/3354166.3354169.

[8] Beniamino Accattoli, Andrea Condoluci, and Claudio Sacerdoti Coen. Strong call-by-value is reasonable, implosively. In *36th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2021, Rome, Italy, June 29 - July 2, 2021*, pages 1–14. IEEE, 2021. doi:10.1109/LICS52264.2021.9470630.

[9] Beniamino Accattoli and Ugo Dal Lago. On the invariance of the unitary cost model for head reduction. In Ashish Tiwari, editor, *23rd International Conference on Rewriting Techniques and Applications (RTA'12) , RTA 2012, May 28 - June 2, 2012, Nagoya, Japan*, volume 15 of *LIPIcs*, pages 22–37. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2012. doi:10.4230/LIPICS.RTA.2012.22.

[10] Beniamino Accattoli and Ugo Dal Lago. (leftmost-outermost) beta reduction is invariant, indeed. *Log. Methods Comput. Sci.*, 12(1), 2016. doi:10.2168/LMCS-12(1:4)2016.

[11] Beniamino Accattoli, Ugo Dal Lago, and Gabriele Vanoni. The (in)efficiency of interaction. *Proc. ACM Program. Lang.*, 5(POPL):1–33, 2021. doi:10.1145/3434332.

[12] Beniamino Accattoli and Giulio Guerrieri. Abstract machines for open call-by-value. *Sci. Comput. Program.*, 184, 2019. doi:10.1016/J.SCICO.2019.03.002.

[13] Beniamino Accattoli, Giulio Guerrieri, and Maico Leberle. Types by need. In Luís Caires, editor, *Programming Languages and Systems - 28th European Symposium on Programming, ESOP 2019, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2019, Prague, Czech Republic, April 6-11, 2019, Proceedings*, volume 11423

of *Lecture Notes in Computer Science*, pages 410–439. Springer, 2019. doi:10.1007/978-3-030-17184-1_15.

[14] Beniamino Accattoli and Delia Kesner. The structural λ-calculus. In Anuj Dawar and Helmut Veith, editors, *Computer Science Logic, 24th International Workshop , CSL 2010, 19th Annual Conference of the EACSL, Brno, Czech Republic, August 23-27, 2010. Proceedings*, volume 6247 of *Lecture Notes in Computer Science*, pages 381–395. Springer, 2010. doi:10.1007/978-3-642-15205-4_30.

[15] Beniamino Accattoli, Ugo Dal Lago, and Gabriele Vanoni. Reasonable space for the λ-calculus, logarithmically. *Log. Methods Comput. Sci.*, 20(4), 2024. doi:10.46298/LMCS-20(4:15)2024.

[16] Beniamino Accattoli and Adrienne Lancelot. Mirroring call-by-need, or values acting silly. In Jakob Rehof, editor, *9th International Conference on Formal Structures for Computation and Deduction, FSCD 2024, July 10-13, 2024, Tallinn, Estonia*, volume 299 of *LIPIcs*, pages 23:1–23:24. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024. doi:10.4230/LIPICS.FSCD.2024.23.

[17] Beniamino Accattoli and Maico Leberle. Useful open call-by-need. In Florin Manea and Alex Simpson, editors, *30th EACSL Annual Conference on Computer Science Logic, CSL 2022, February 14-19, 2022, Göttingen, Germany*, volume 216 of *LIPIcs*, pages 4:1–4:21. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022. doi:10.4230/LIPICS.CSL.2022.4.

[18] Beniamino Accattoli, Francesco Magliocca, Loïc Peyrot, and Claudio Sacerdoti Coen. The cost of skeletal call-by-need, smoothly, 2025. arXiv:2505.09242.

[19] Beniamino Accattoli and Claudio Sacerdoti Coen. On the value of variables. *Inf. Comput.*, 255:224–242, 2017. doi:10.1016/J.IC.2017.01.003.

[20] Beniamino Accattoli and Claudio Sacerdoti Coen. IMELL cut elimination with linear overhead. In Jakob Rehof, editor, *9th International Conference on Formal Structures for Computation and Deduction, FSCD 2024, July 10-13, 2024, Tallinn, Estonia*, volume 299 of *LIPIcs*, pages 24:1–24:24. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024. URL: https://doi.org/10.4230/LIPIcs.FSCD.2024.24, doi:10.4230/LIPICS.FSCD.2024.24.

[21] Zena M. Ariola, Matthias Felleisen, John Maraist, Martin Odersky, and Philip Wadler. The call-by-need lambda calculus. In Ron K. Cytron and Peter Lee, editors, *Conference Record of POPL'95: 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, San Francisco, California, USA, January 23-25, 1995*, pages 233–246. ACM Press, 1995. doi:10.1145/199448.199507.

[22] Thibaut Balabonski. A unified approach to fully lazy sharing. In John Field and Michael Hicks, editors, *Proceedings of the 39th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2012, Philadelphia, Pennsylvania, USA, January 22-28, 2012*, pages 469–480. ACM, 2012. doi:10.1145/2103656.2103713.

[23] Thibaut Balabonski. Weak optimality, and the meaning of sharing. In Greg Morrisett and Tarmo Uustalu, editors, *ACM SIGPLAN International Conference on Functional Programming, ICFP'13, Boston, MA, USA - September 25 - 27, 2013*, pages 263–274. ACM, 2013. doi:10.1145/2500365.2500606.

[24] Thibaut Balabonski, Pablo Barenbaum, Eduardo Bonelli, and Delia Kesner. Foundations of strong call by need. *PACMPL*, 1(ICFP):20:1–20:29, 2017. doi:10.1145/3110264.

[25] Thibaut Balabonski, Antoine Lanco, and Guillaume Melquiond. A strong call-by-need calculus. In Naoki Kobayashi, editor, *6th International Conference on Formal Structures for Computation and Deduction, FSCD 2021, July 17-24, 2021, Buenos Aires, Argentina (Virtual Conference)*, volume 195 of *LIPIcs*, pages 9:1–9:22. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021. doi:10.4230/LIPICS.FSCD.2021.9.

[26] Pablo Barenbaum, Eduardo Bonelli, and Kareem Mohamed. Pattern matching and fixed points: Resource types and strong call-by-need: Extended abstract. In David Sabel and Peter Thiemann, editors, *Proceedings of the 20th International Symposium on Principles and Practice of Declarative Programming, PPDP 2018, Frankfurt am Main, Germany, September 03-05, 2018*, pages 6:1–6:12. ACM, 2018. doi:10.1145/3236950.3236972.

[27] Andrea Condoluci, Beniamino Accattoli, and Claudio Sacerdoti Coen. Sharing equality is linear. In Ekaterina Komendantskaya, editor, *Proceedings of the 21st International Symposium on Principles and Practice of Programming Languages, PPDP 2019, Porto, Portugal, October 7-9, 2019*, pages 9:1–9:14. ACM, 2019. doi:10.1145/3354166.3354174.

[28] Yannick Forster, Fabian Kunze, and Marc Roth. The weak call-by-value λ-calculus is reasonable for both time and space. *Proc. ACM Program. Lang.*, 4(POPL):27:1–27:23, 2020. doi:10.1145/3371095.

[29] Tom Gundersen, Willem Heijltjes, and Michel Parigot. Atomic lambda calculus: A typed lambda-calculus with explicit sharing. In *28th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2013, New Orleans, LA, USA, June 25-28, 2013*, pages 311–320. IEEE Computer Society, 2013. doi:10.1109/LICS.2013.37.

[30] John Hughes. *The Design and Implementation of Programming Languages*. PhD thesis, Programming Research Group, Oxford University, July 1983.

[31] Delia Kesner. Reasoning about call-by-need by means of types. In Bart Jacobs and Christof Löding, editors, *Foundations of Software Science and Computation Structures - 19th International Conference, FOSSACS 2016, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, April 2-8, 2016, Proceedings*, volume 9634 of *Lecture Notes in Computer Science*, pages 424–441. Springer, 2016. doi:10.1007/978-3-662-49630-5_25.

[32] Delia Kesner, Loïc Peyrot, and Daniel Ventura. Node replication: Theory and practice. *Log. Methods Comput. Sci.*, 20(1), 2024. doi:10.46298/LMCS-20(1:5)2024.

[33] Delia Kesner, Alejandro Ríos, and André s Viso. Call-by-need, neededness and all that. In Christel Baier and Ugo Dal Lago, editors, *Foundations of Software Science and Computation Structures - 21st International Conference, FOSSACS 2018, Thessaloniki, Greece, April 14-20, 2018 , Proceedings*, volume 10803 of *Lecture Notes in Computer Science*, pages 241–257. Springer, 2018. doi:10.1007/978-3-319-89366-2_13.

[34] Arne Kutzner and Manfred Schmidt-Schauß. A non-deterministic call-by-need lambda calculus. In Matthias Felleisen, Paul Hudak, and Christian Queinnec, editors, *Proceedings of the third ACM SIGPLAN International Conference on Functional Programming (ICFP '98), Baltimore, USA, September 27-29, 1998*, pages 324–335. ACM, 1998. doi:10.1145/289423.289462.

[35] John Launchbury. A natural semantics for lazy evaluation. In Mary S. Van Deusen and Bernard Lang, editors, *Conference Record of the Twentieth Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Charleston, South Carolina, USA, January 1993*, pages 144–154. ACM Press, 1993. doi:10.1145/158511.158618.

[36] Simon L. Peyton Jones. *The Implementation of Functional Programming Languages*. Prentice-Hall, 1987.

[37] David Sands, Jörgen Gustavsson, and Andrew Moran. Lambda calculi and linear speedups. In Torben Æ. Mogensen, David A. Schmidt, and Ivan Hal Sudborough, editors, *The Essence of Computation, Complexity, Analysis, Transformation. Essays Dedicated to Neil D. Jones [on occasion of his 60th birthday]*, volume 2566 of *Lecture Notes in Computer Science*, pages 60–84. Springer, 2002. doi:10.1007/3-540-36377-7_4.

[38] Peter Sestoft. Deriving a lazy abstract machine. *J. Funct. Program.*, 7(3):231–264, 1997. doi:10.1017/S0956796897002712.

[39] Olin Shivers and Mitchell Wand. Bottom-up beta-reduction: Uplinks and lambda-dags. *Fundam. Informaticae*, 103(1-4):247–287, 2010. doi:10.3233/FI-2010-328.

[40] D. A. Turner. A new implementation technique for applicative languages. *Softw. Pract. Exp.*, 9(1):31–49, 1979. doi:10.1002/SPE.4380090105.

[41] Christopher P. Wadsworth. *Semantics and pragmatics of the lambda-calculus*. PhD Thesis, Oxford, 1971.